

SALBP-2

ALESSANDRO LORI - GIADA PICA
PROGETTO DI AMOD



TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA



Introduzione al problema

SALBP - SIMPLE ASSEMBLY LINE BALANCING PROBLEM

Parliamo di un problema NP-Hard

Caratterizzato da:

- un numero di **task** e vincoli di precedenza
- un numero di **stazioni** in sequenza
- obiettivo di assegnazione di ogni task ad una stazione rispettando le precedenze ed il **cycle time C**

2 Varianti classiche



SALBP-1

Dato **C**, ha come obiettivo minimizzare **M**

SALBP-2

Dato **M**, ha come obiettivo minimizzare **C**

Salbp-2 NP-Hard

Il problema di bilanciamento delle linee di montaggio è noto per essere un problema **NP-Hard**, ovvero la difficoltà di risolverlo in tempi ragionevoli aumenta esponenzialmente al crescere delle istanze tramite algoritmi deterministici.

Di conseguenza, si ricorre spesso a metodi euristici o metaeuristici.



Istanze e dati

SCHOLL 1993

Le principali stanze utilizzate sono quelle studiate da Scholl e descritte nel **pdf** allegato in basso.

Si sono ulteriormente eseguiti test su istanze generate che non hanno portato a risultati significativi data la semplicità di esse.

I grafici che vedremo sono stati eseguiti con dataset **Scholl** puro, istanza di **297 task** e con $m = 25$ o 15 , per dare risultati su un benchmark pesante.

Data of Assembly Line Balancing Problems

Istanze e dati

Immagine presa dal pdf precedentemente citato pag. 14

Name	<u>m</u>	$\bar{m}$	idle*	m*	c*
Scholl	3	4	1	4	17414
	6	51	1	8	8707
	9	20	5	9	7740
	10	20	5	10	6966
	13	14	9	14	4976
	16	17	9	16	435
	19	20	5	20	3483
	22	28	5	27	2580
	22	51	2	31	2247
	25	26	20	25	2787
	32	40	5-8	33	2111
	34	35	11	34	2049
	38	39	37	38	1834
	42	51	23-306	43	1627
	44	48	35-481	44	1594
	47	48	41-563	47	1494

Euristiche e meta-euristiche

EURISTICHE

Le euristiche sono metodi di risoluzione dei problemi che utilizzano un approccio pratico per trovare soluzioni sufficientemente buone in un tempo ragionevole, senza garantire l'ottimalità.

META - EURISTICHE

Le meta-euristiche rappresentano un approccio per tentativi ed errori, che utilizza un framework di metodi euristici, muovendosi tra più soluzioni decidendo se accettare o rifiutare le mosse applicate per trovare un miglioramento locale.

Euristiche

TARGET-C

È uno schema euristico in cui si cerca il Cycle Time minimo C calcolato come lower bound, tenendo conto del task più lungo e del carico medio calcolato come rapporto tra somma dei tempi e numero stazioni, per costruire una soluzione fattibile, se fallisce il C minimo viene incrementato di un valore predefinito finché la costruzione non va a buon fine.

COSTRUTTIVA GREEDY - RPW

COSTRUTTIVA GREEDY - LPT

COSTRUTTIVA GREEDY BEST-FIT

Euristiche

GREEDY-RPW:

Costruisce una soluzione Greedy ordinando i task per Ranked Poitional Weight:

- Per ogni task i calcolo il peso posizionale $RPW(i)$:
 $RPW(i) = \text{tempo}(i) + \text{somma dei tempi di tutti i suoi successori}.$
- Quando deve scegliere il prossimo task:
Prende il task con RPW più alto.
Prova a metterlo nella stazione corrente entro il valore di C .
Se non è entro il valore C passa alla stazione successiva.
Se non rimangono più stazioni C viene aumentato e ripete il procedimento

Da priorità ai task più critici per la rete di precedenza.

Euristiche

GREEDY-LPT:

Costruisce una soluzione Greedy ordinando i task per Longest Process Time:

- Ordina i task per tempo decrescente.
- Procede stazione per stazione:
 - Prova ad assegnare il task più lungo alla stazione corrente.
 - Se rispetta la capienza di stazione lo mette lì.
 - Se non ci sta passa alla stazione successiva.
 - Continua fino ad esaurimento task o stazioni.
 - Se fallisce C viene aumentato e ripete il procedimento.

Euristiche

GREEDY-BEST FIT:

Costruisce una soluzione Greedy di tipo Bin-Packing con strategia Best-Fit:

- Per ogni task:

Considera tutte le stazioni in cui può stare

Sceglie la stazione nella quale resterebbe più spazio dopo aver messo il task

Se non esiste alcuna stazione valida fallisce, incrementa C e ripete il procedimento



Meta - Euristiche

Tecniche di local-search usate come post-processing al fine di migliorare iterativamente una soluzione già esistente. Vengono applicate determinate mosse locali scelte all'interno di un vicinato della soluzione corrente cercando di passare ad una soluzione vicina migliorante cercando così un miglioramento locale.

Un vicinato è l'insieme delle soluzioni che si possono ottenere partendo da una soluzione precedente alla quale viene applicata una determinata mossa.

VND: VARIABLE NEIGHBORHOOD DESCENT

Meta - Euristiche

1-move:

Sposta una task da una stazione ad un'altra cercando di migliorare il cycle time C o almeno di bilanciare meglio i carichi sulle stazioni:

- Per ogni task i :
 - Sposta da stazione vecchia a stazione nuova.
 - Se il cycle time C o il bilanciamento migliora accetta lo spostamento.
- Conclude quando non ci sono più mosse valide o scade il tempo di esecuzione.

Meta - Euristiche

Variable Neighborhood Descent:

Utilizza più tipi di mosse, nei relativi vicinati:

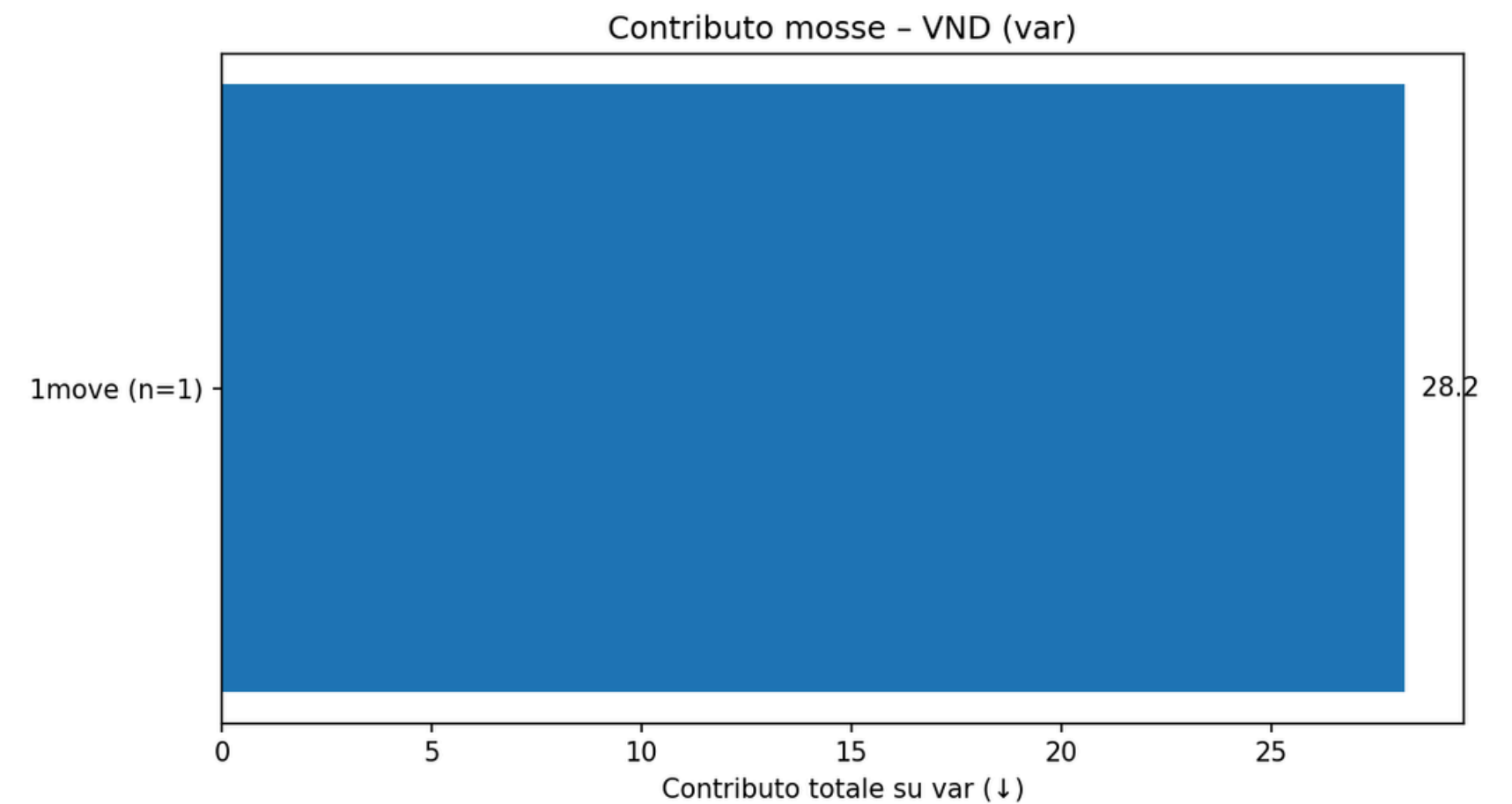
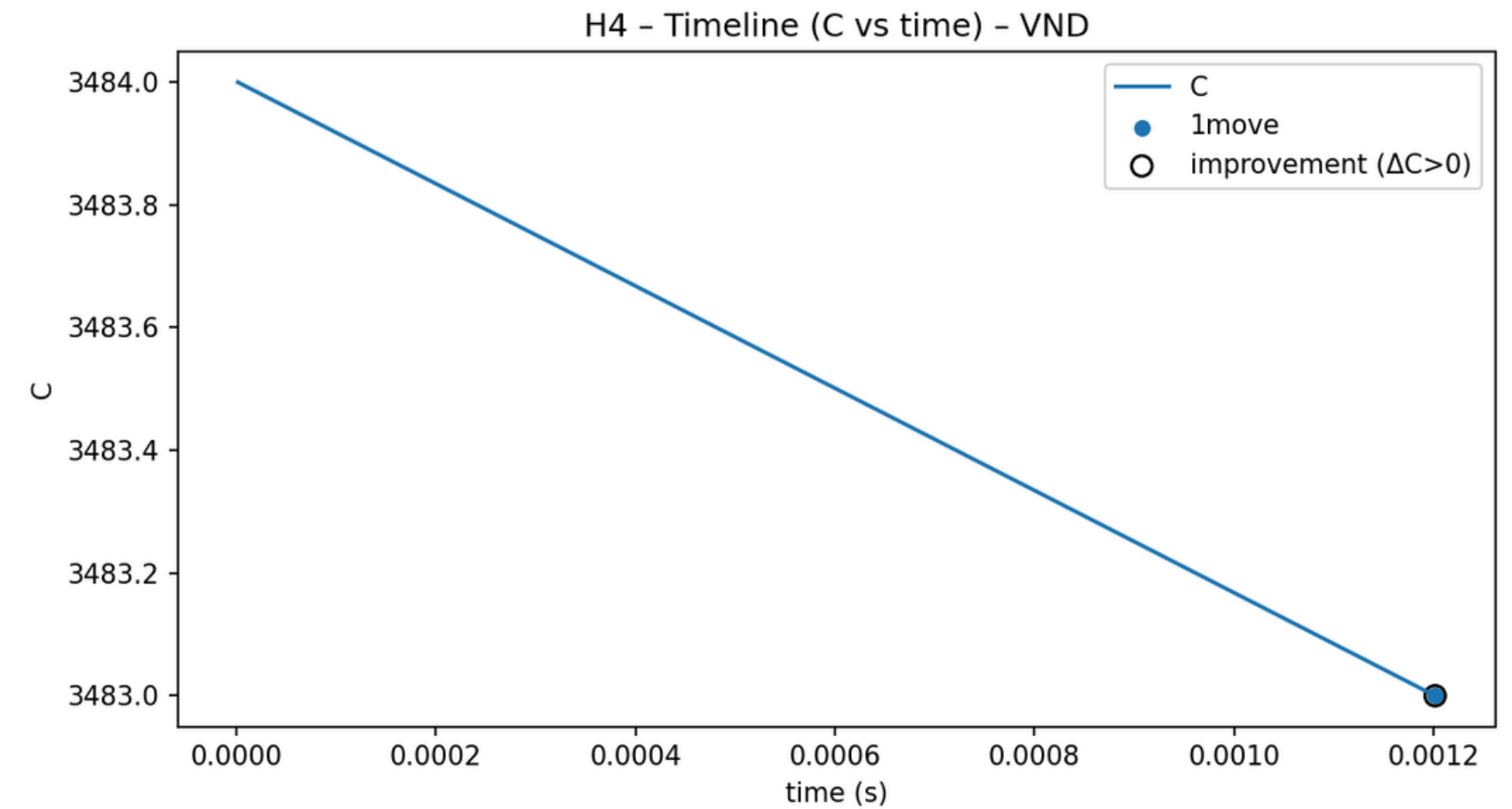
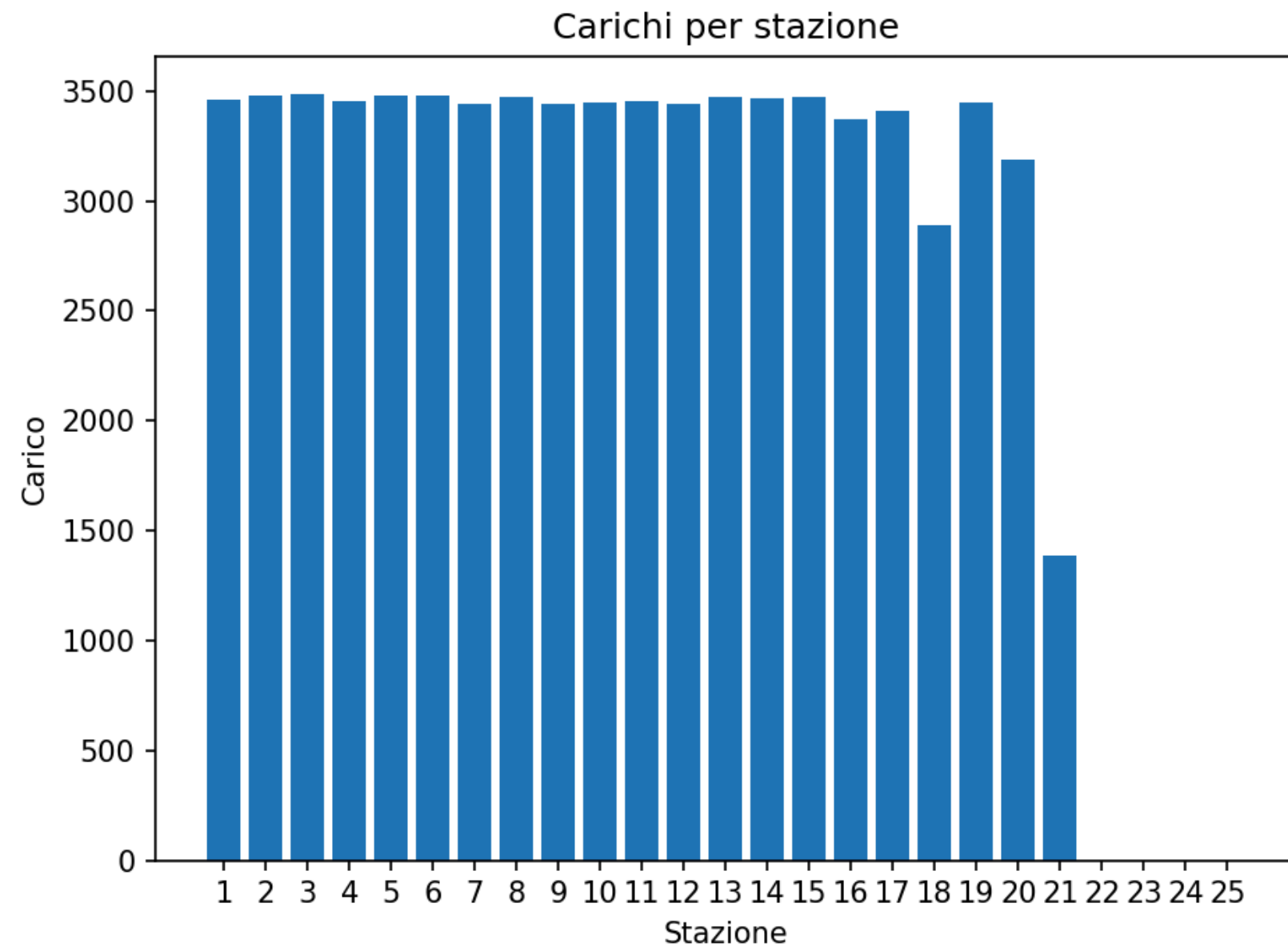
- Parte applicando la 1-Move finché trova miglioramenti.
- Quando non trova più miglioramenti applica la Swap scambiando delle task tra due stazioni, finché non trova un miglioramento, per poi tornare alla 1-Move.
- Se non trova miglioramenti applica l'Ejection ricollocando alcune delle task di una stazione più carica, finché non trova un miglioramento, per poi tornare alla 1-Move.
- Termina quando nessuna mossa è più disponibile.

Possiamo applicare un tie-break di tipo range o di tipo var:

- Range: date le stazioni con carico massimo e carico minimo opera per andare a ridurre il valore della distanza tra carichi.
- Var: data la varianza di carico nel sistema opera per andare a ridurre tale valore il più vicino possibile al valore medio di carico del sistema.

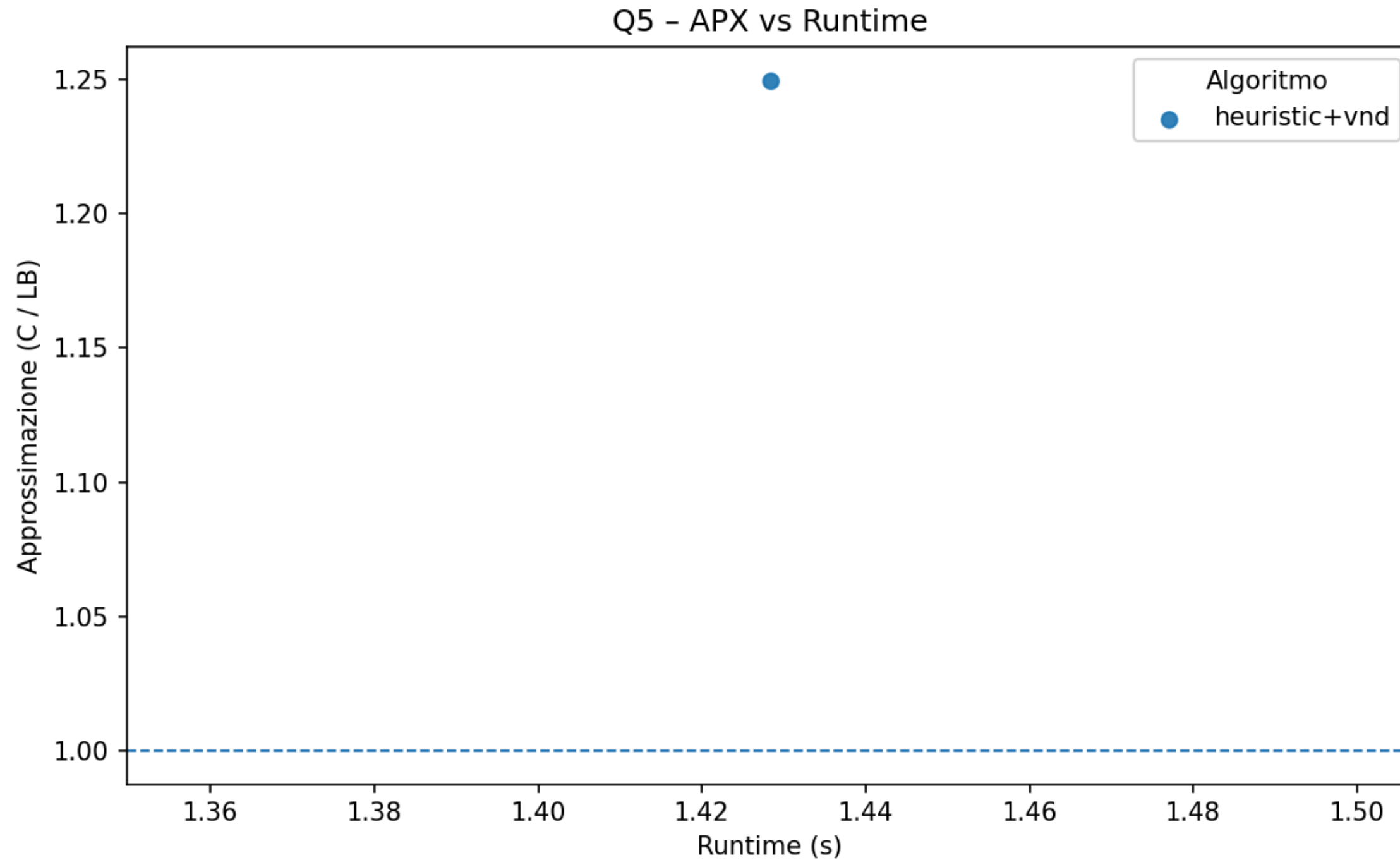
Grafici euristiche

Best-Fit local search tie-break Var



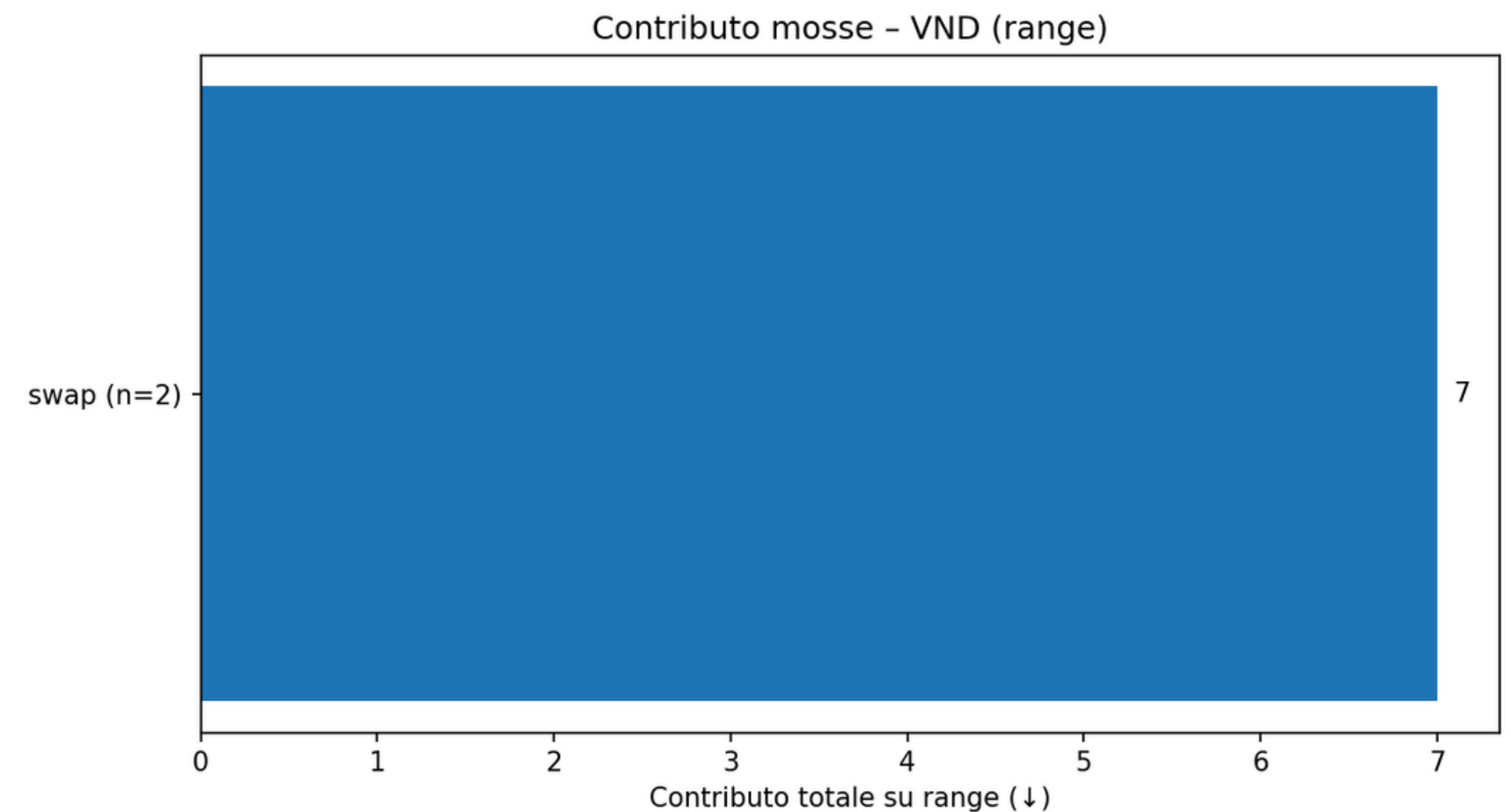
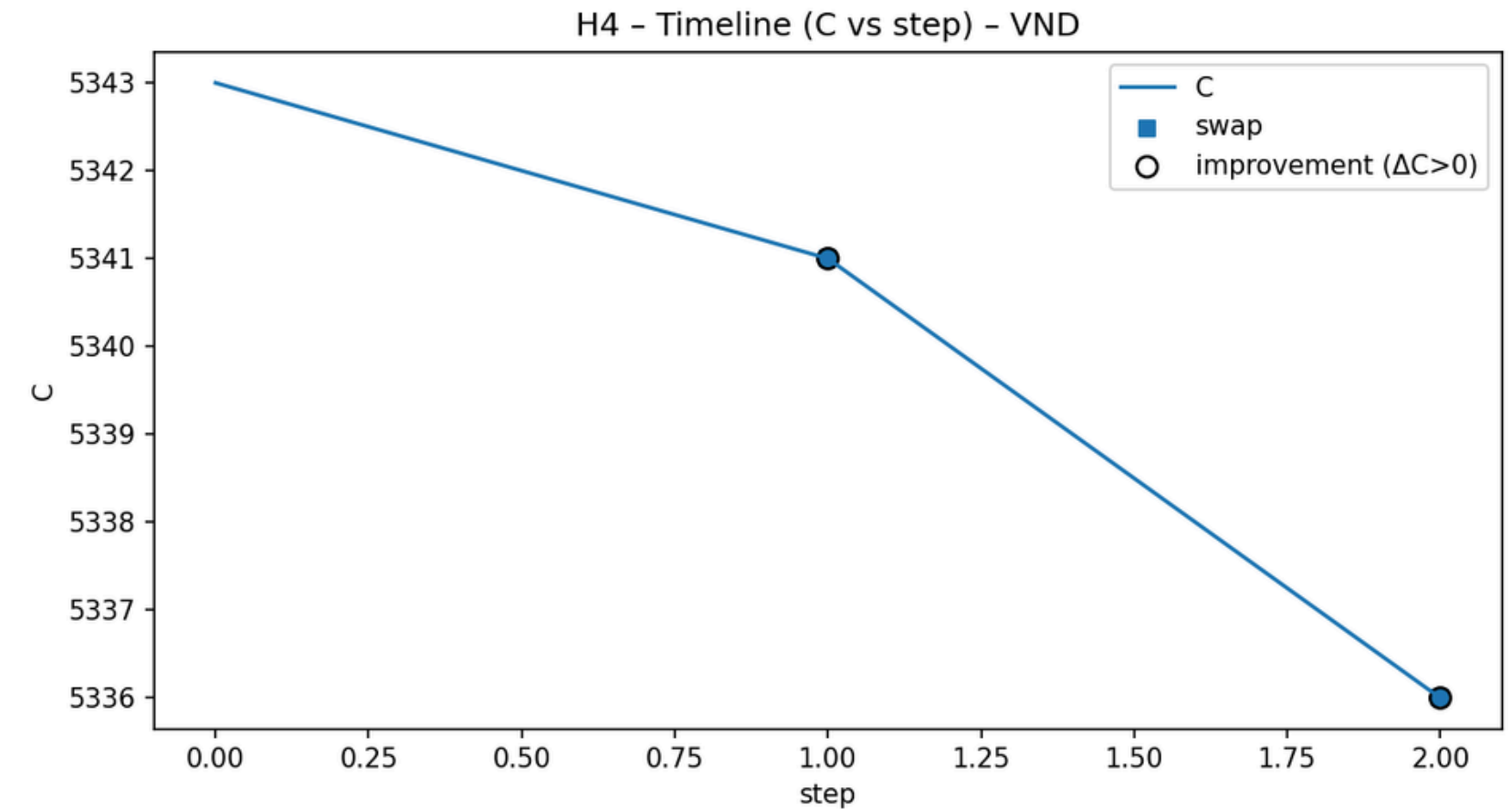
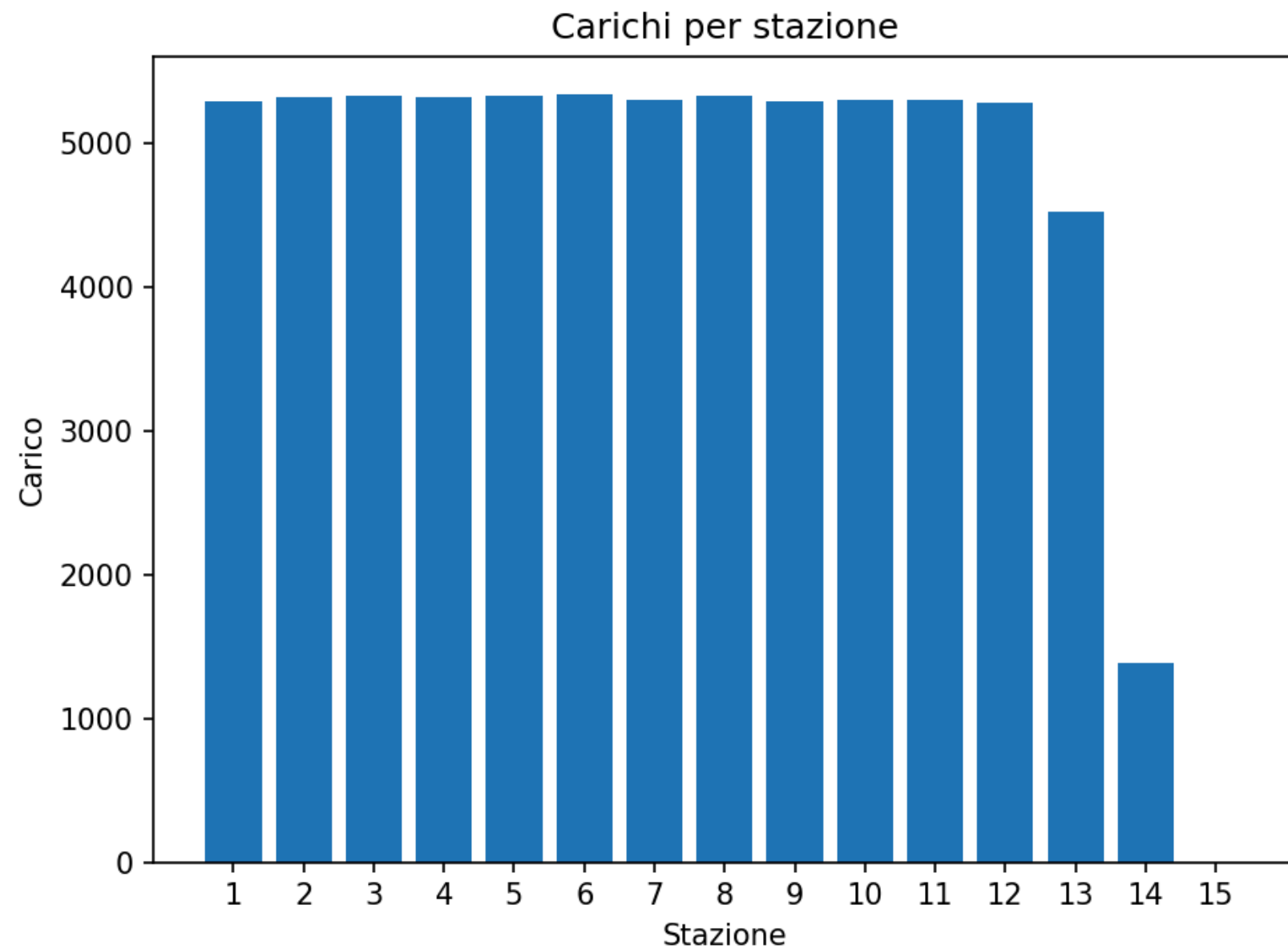
Grafici euristiche

Best-Fit local search tie-break Var:



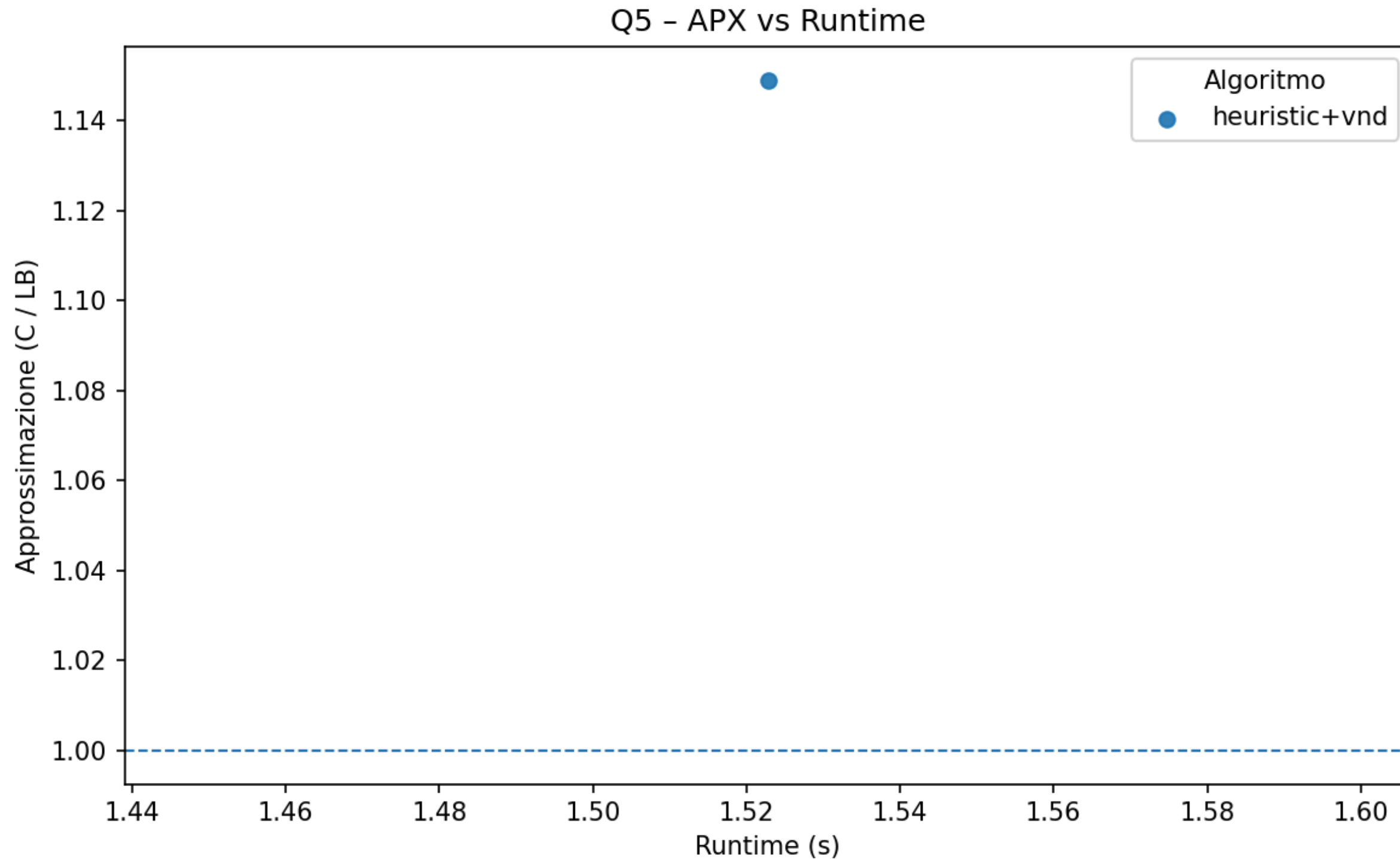
Grafici euristiche

Best-Fit local search tie-break Range:



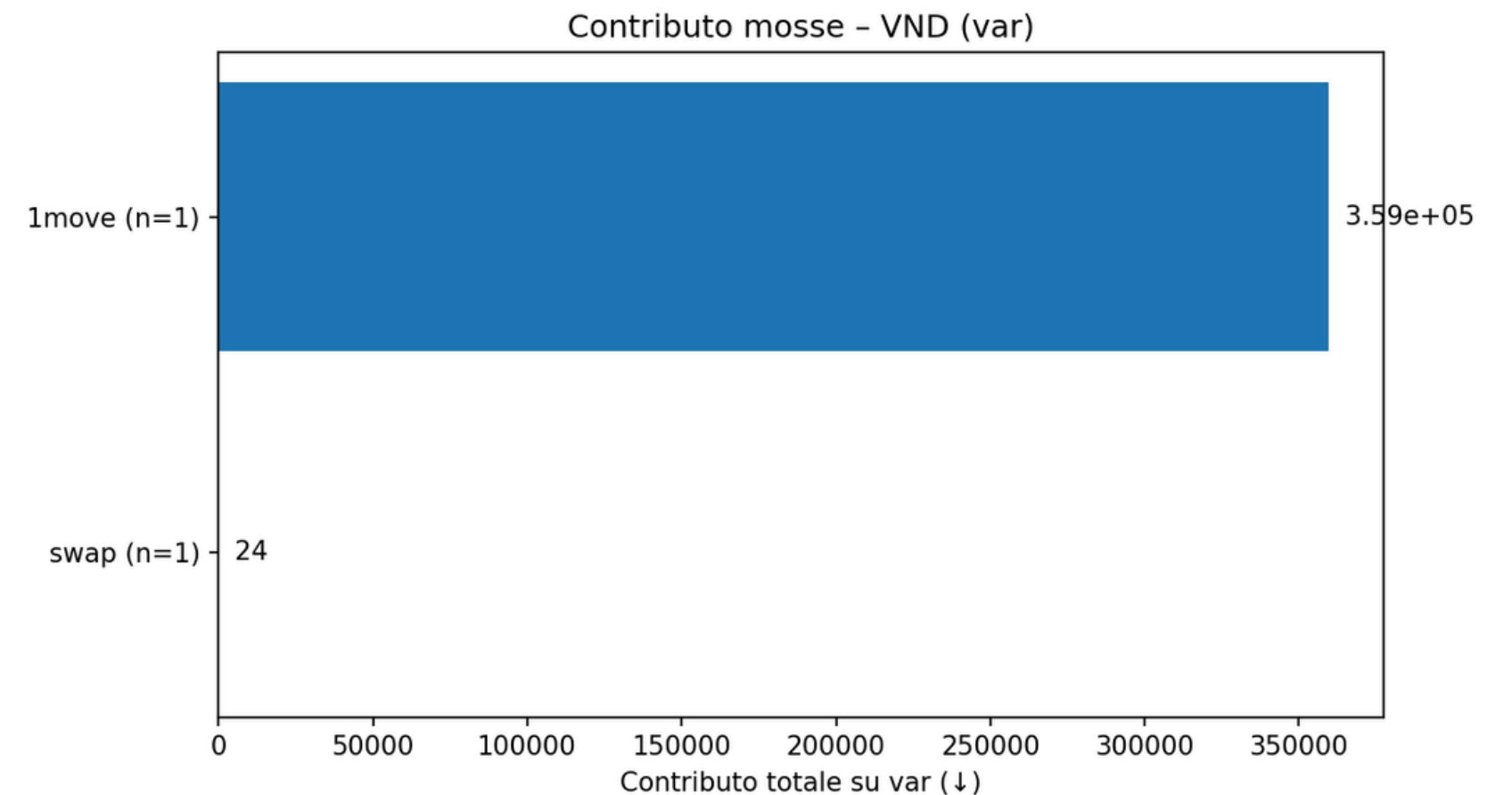
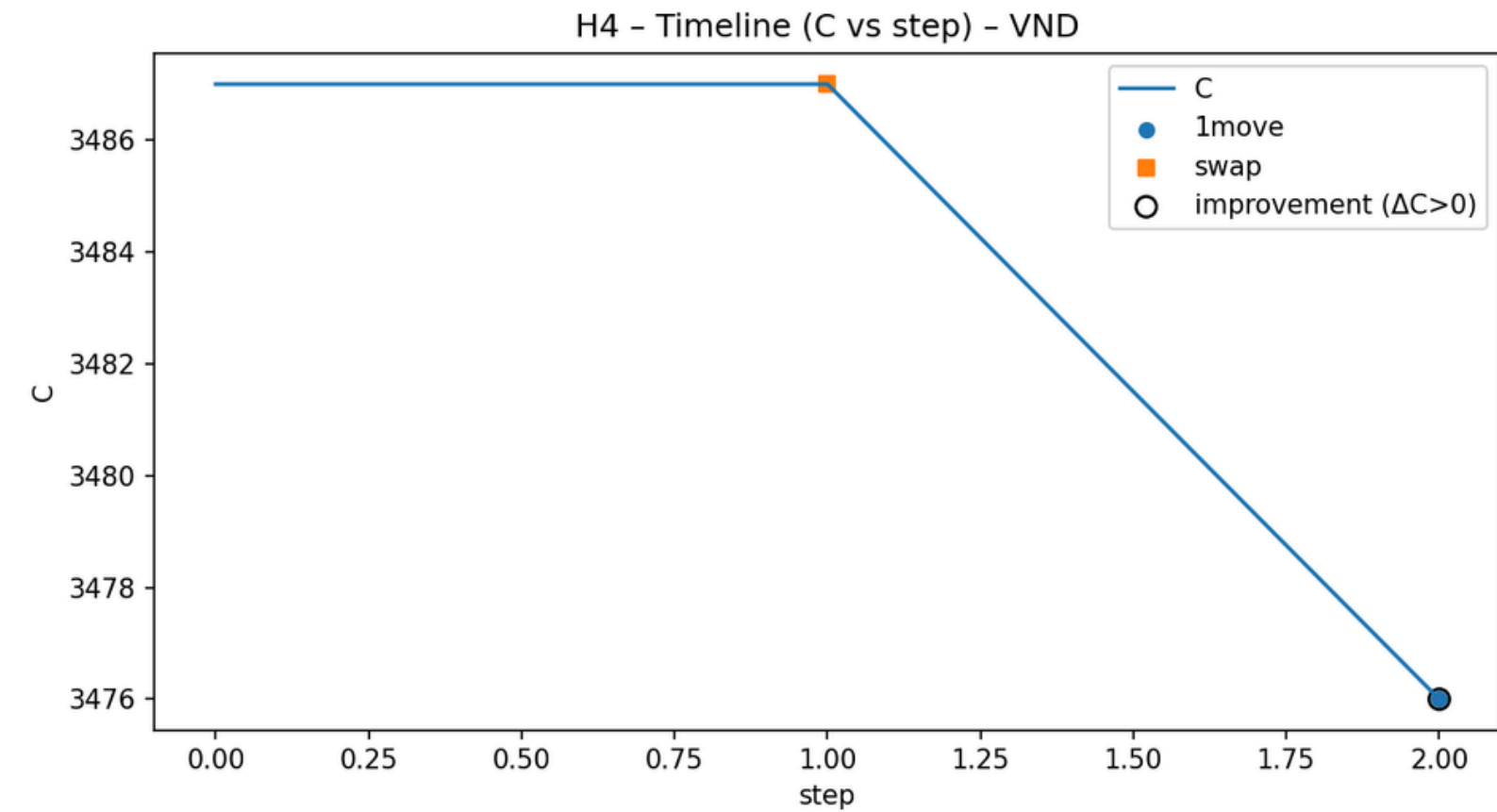
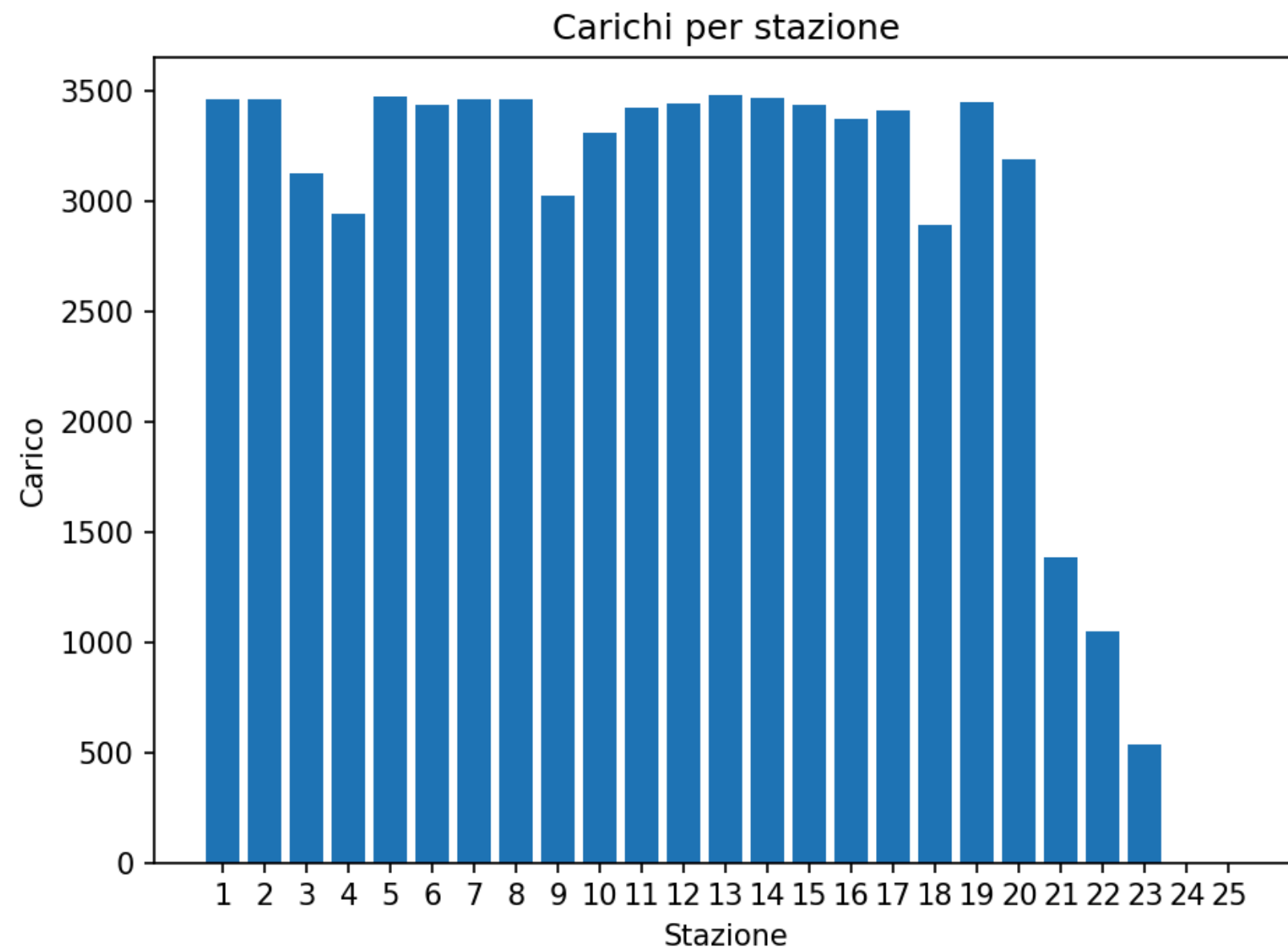
Grafici euristiche

Best-Fit local search tie-break Range:



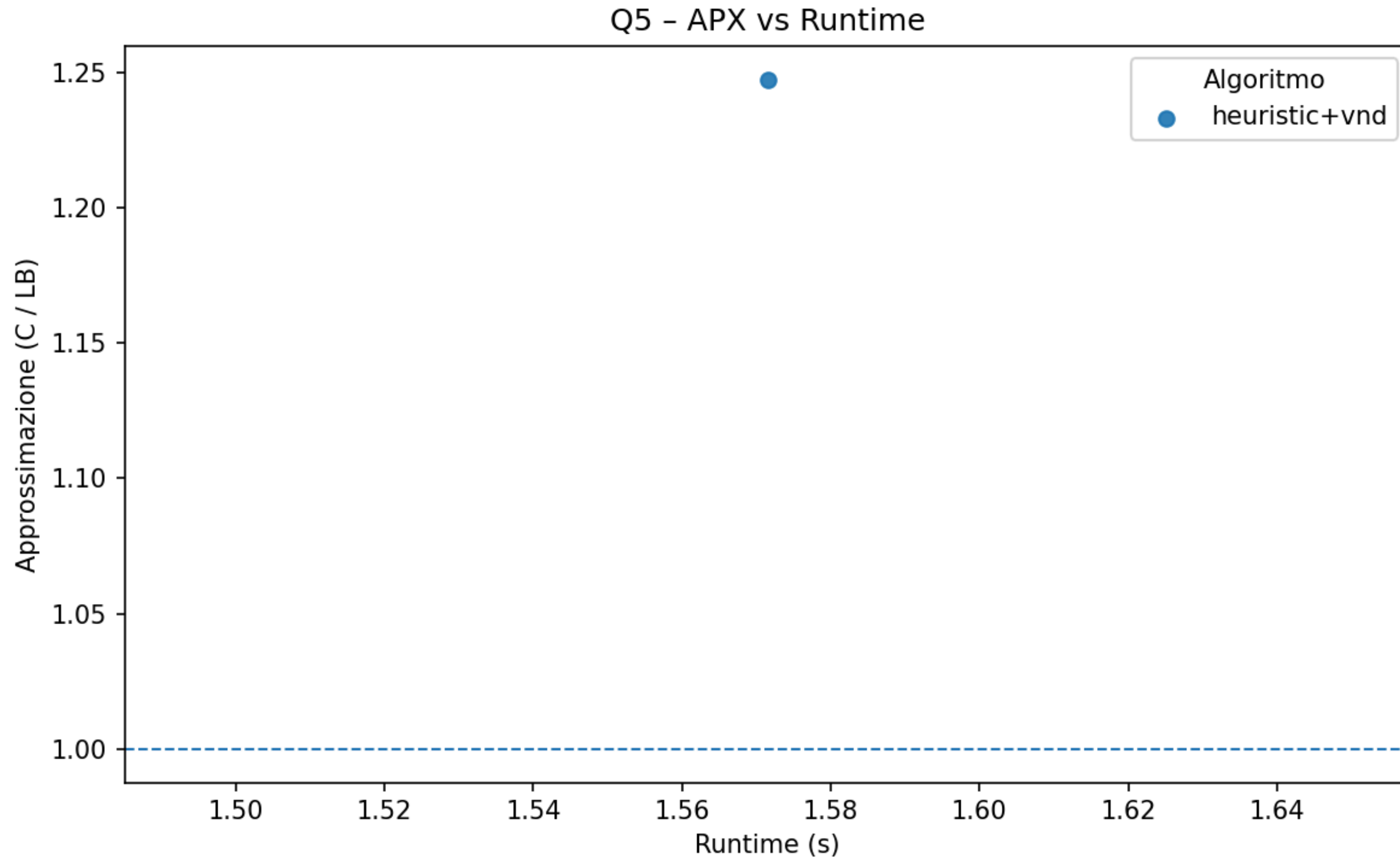
Grafici euristiche

LPT local search tie-break Var:



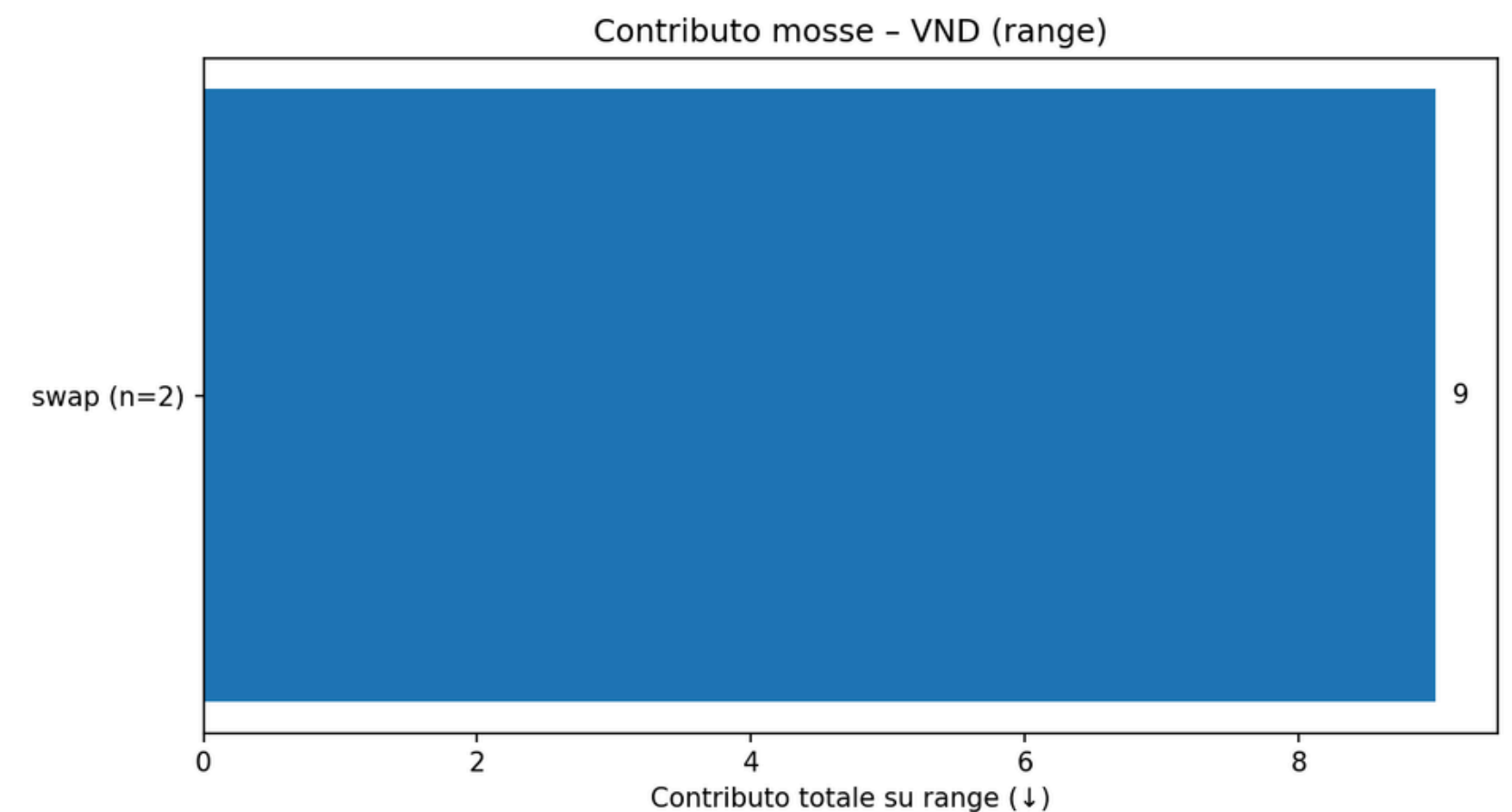
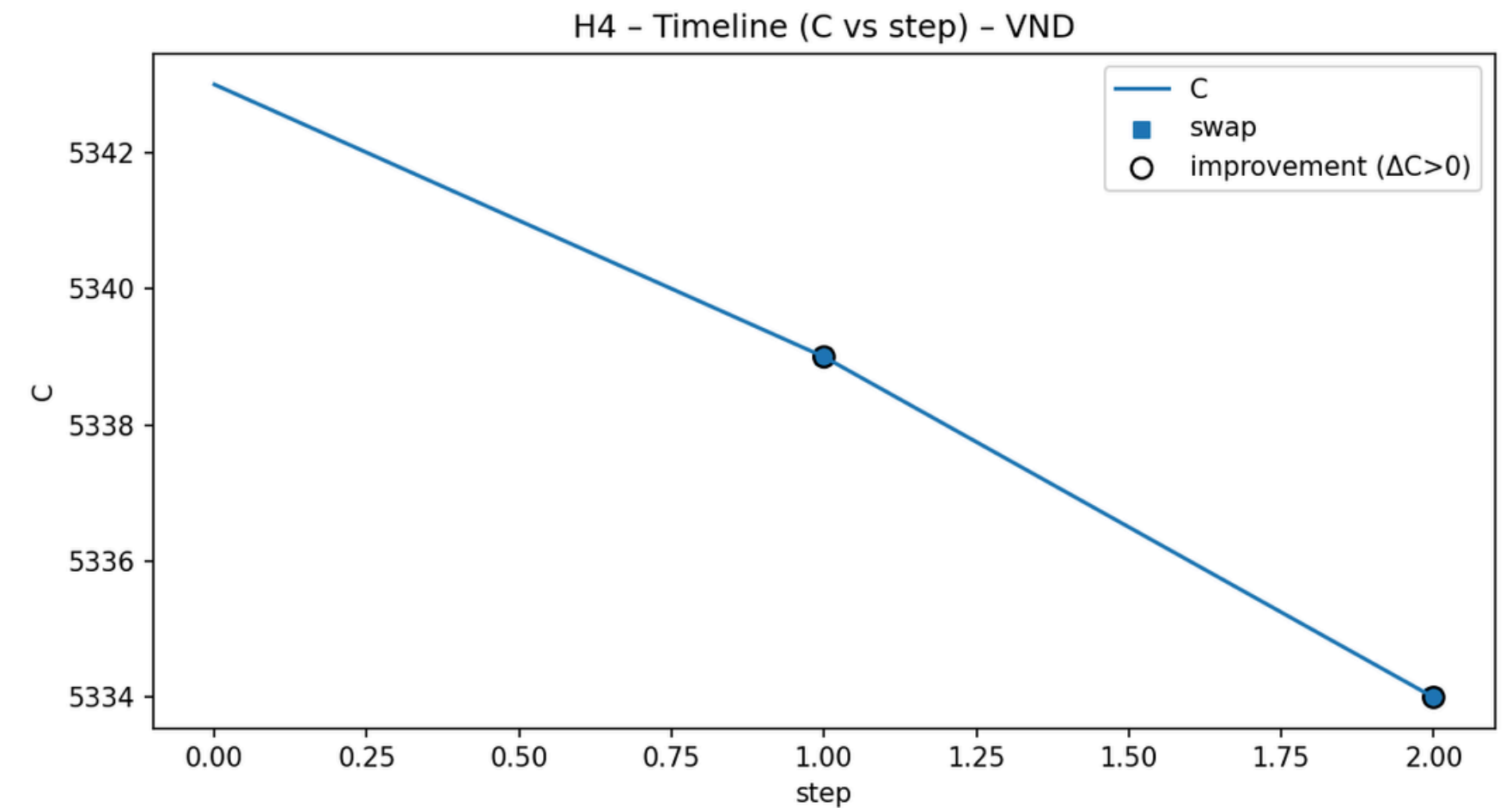
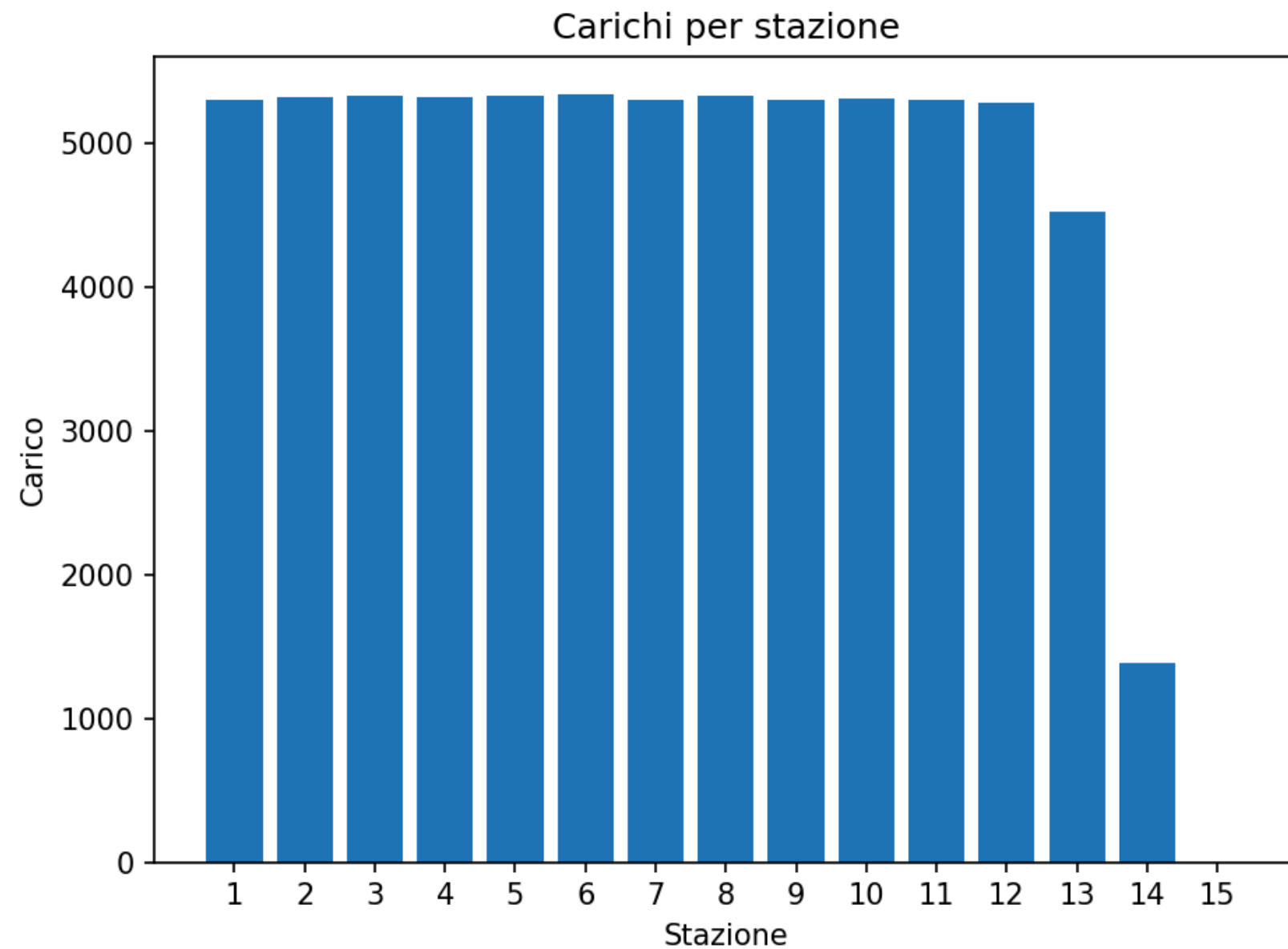
Grafici euristiche

LPT local search tie-break Var:



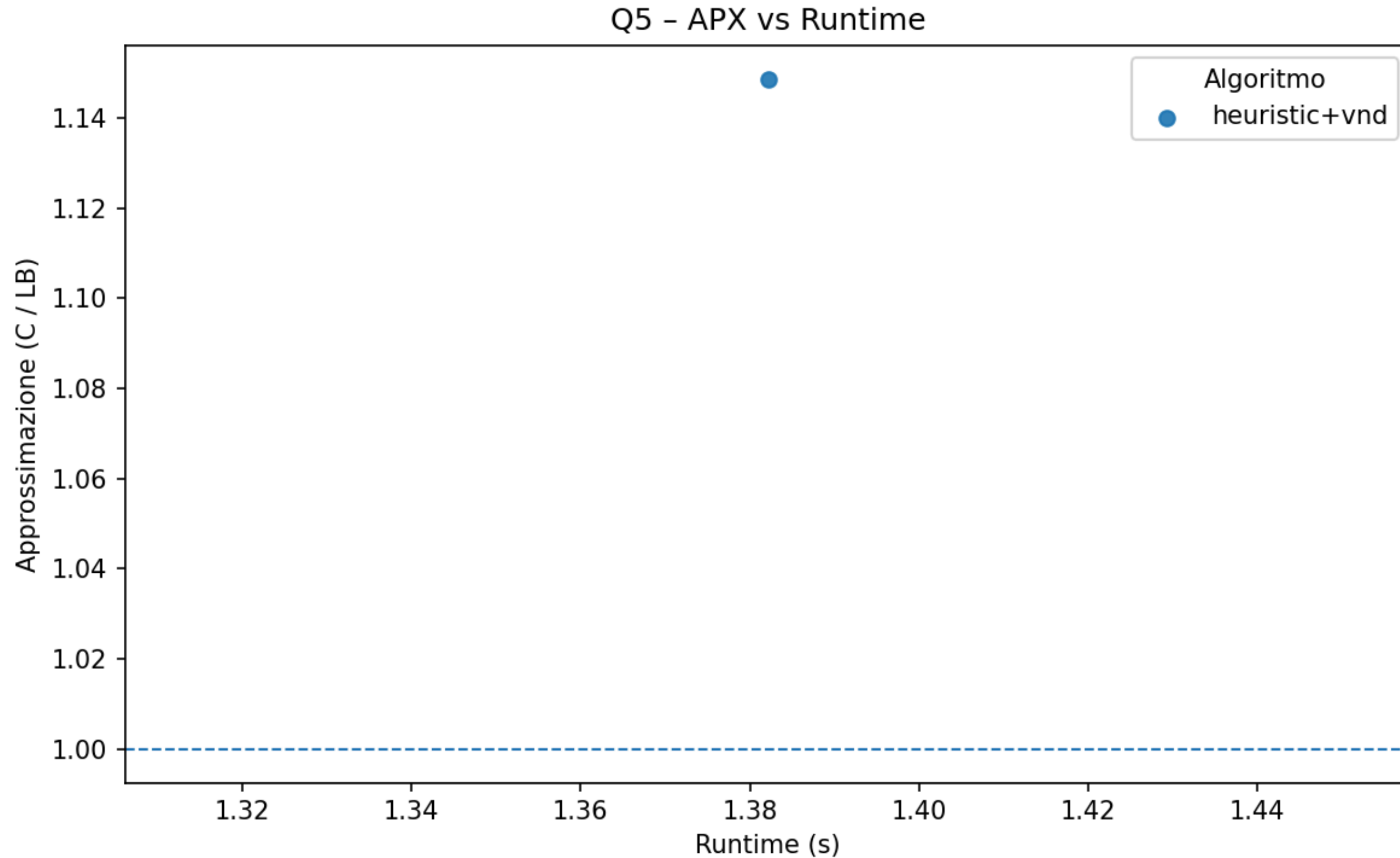
Grafici euristiche

LPT local search tie-break Range:



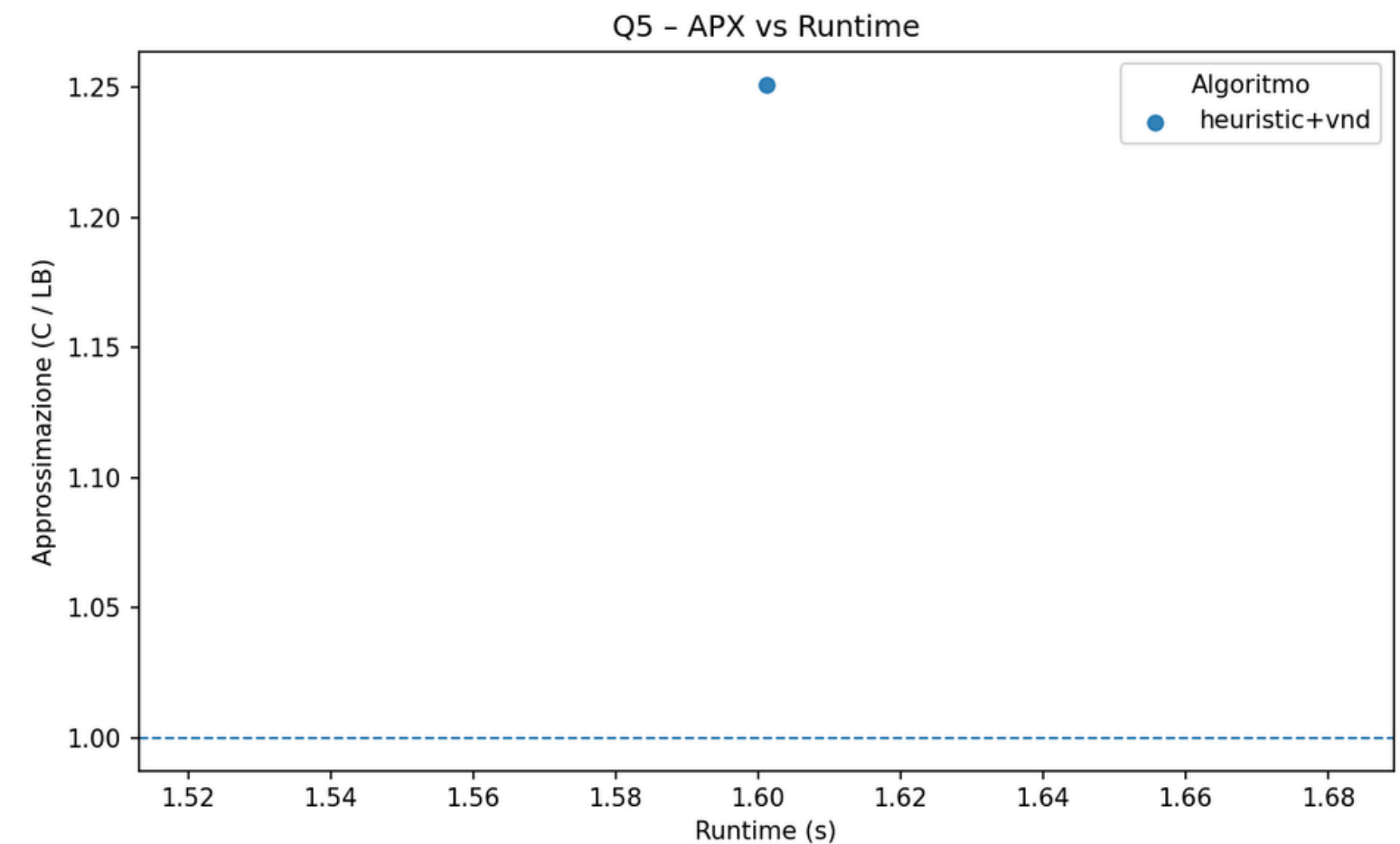
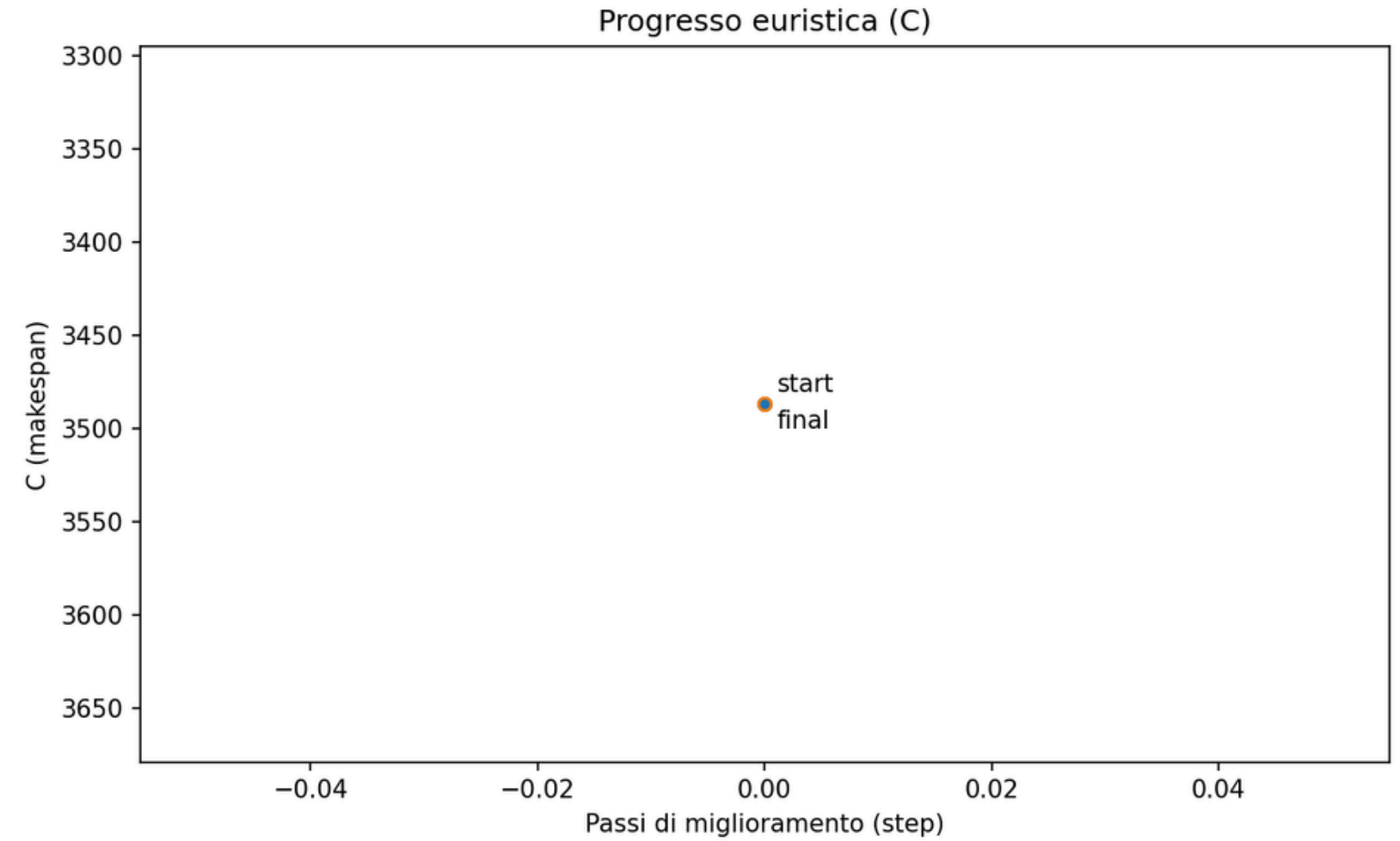
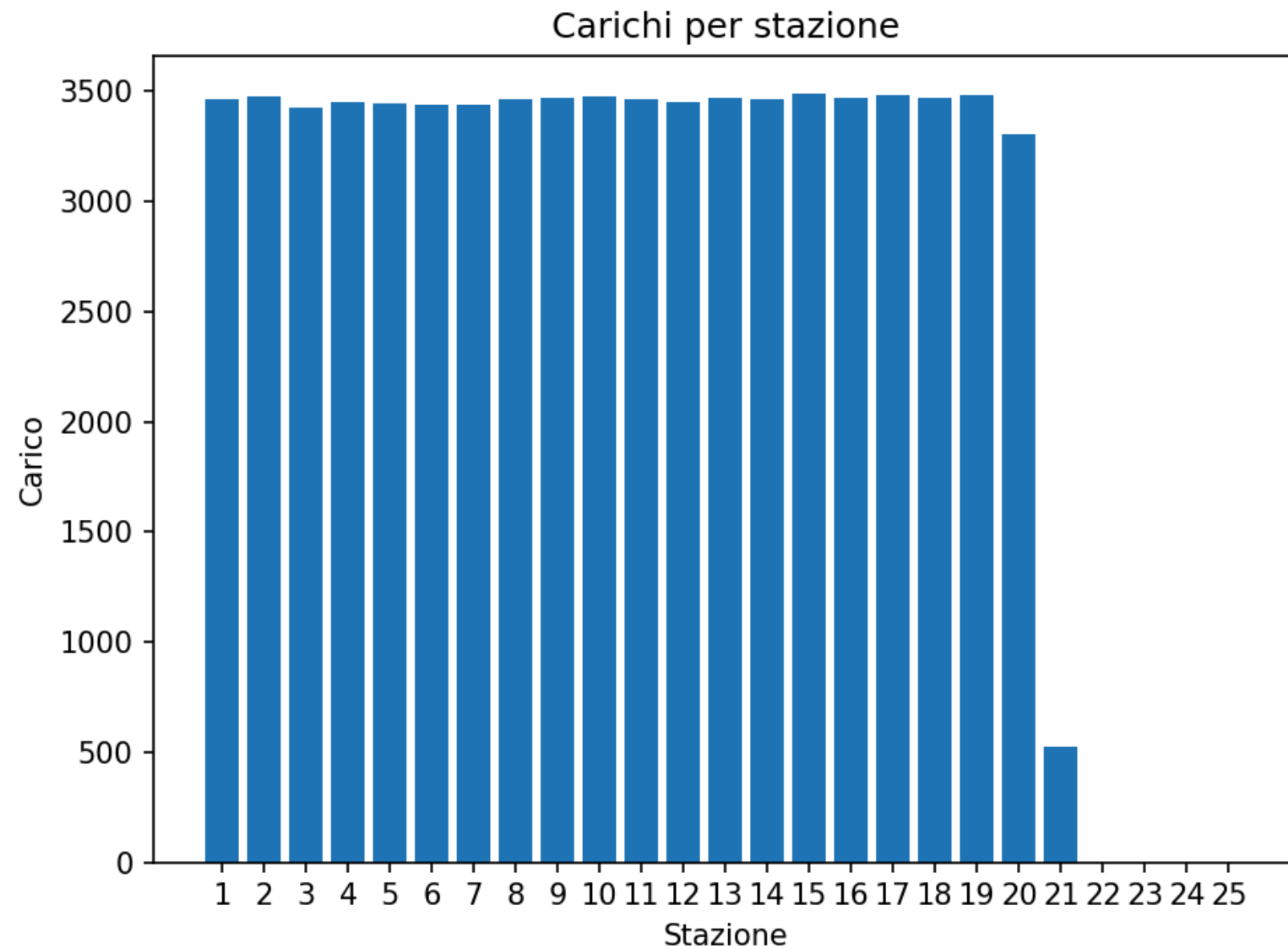
Grafici euristiche

LPT local search tie-break Range:



Grafici euristiche

RPW local search tie-break Var:



Formulazioni MIP

MIP è una tecnica di ottimizzazione matematica che mira a trovare e certificare l'**ottimo globale**.

I solver combinano rilassamenti LP con branch-and-bound/branch-and-cut, fornendo limiti globali; quando il MIP **gap** = **0%**, la soluzione intera è provata ottima (certificazione di ottimalità).

Formulazione MILP - formulazione y

Variabili

- $x_{is} \in \{0,1\}$ per $i \in N, s \in S$: vale 1 se il task i è assegnato alla stazione s
- $y_i \in \{1, \dots, M\}$ per $i \in N$: indice della stazione a cui è assegnato il task i
- $L_s \geq 0$ per $s \in S$: carico di lavoro (somma dei tempi) della stazione s
- $C \geq 0$: **massimo** carico tra le stazioni

Modello

min C

$$\begin{aligned} \text{s.t. } \sum_{s \in S} x_{is} &= 1 && \forall i \in N \quad (\text{assegnamento unico}) \\ y_i &= \sum_{s \in S} s x_{is} && \forall i \in N \quad (\text{collegamento indice stazione}) \\ y_j &\geq y_i && \forall (i, j) \in \mathcal{P} \quad (\text{rispetto delle precedenze}) \\ L_s &= \sum_{i \in N} t_i x_{is} && \forall s \in S \quad (\text{definizione carico di stazione}) \\ L_s &\leq C && \forall s \in S \quad (C \text{ è il massimo carico}) \\ x_{is} &\in \{0, 1\}, \quad y_i \in \{1, \dots, M\}, \quad L_s \geq 0, \quad C \geq 0 \end{aligned}$$

- N numero dei task
- S insieme di stazioni va da 1 a M
- t_i è un numero positivo che indica il tempo di completamento del task i
- l'insieme $\mathcal{P}$ contiene le precedenze ij del grafo

Formulazione MIP - formulazione prefix

Variabili: $x_{is} \in \{0, 1\}$ per $i \in N, s \in S$ (assegnamento)
 $u_{is} \in [0, 1]$ per $i \in N, s \in S$ (prefisso: 1 se stazione(i) $\leq s$)
 $C \geq 0$ (cycle time)

Modello

min C

$$\sum_{s \in S} x_{is} = 1 \quad \forall i \in N \quad (\text{ogni task in una sola stazione})$$

$$u_{i1} = x_{i1} \quad \forall i \in N \quad (\text{prefisso alla prima stazione})$$

$$u_{is} = u_{i,s-1} + x_{is} \quad \forall i \in N, \forall s = 2, \dots, M \quad (\text{costruzione del prefisso})$$

$$u_{is} \geq u_{js} \quad \forall (i \rightarrow j) \in \mathcal{P}, \forall s \in S \quad (\text{precedenze: } i \text{ non può essere dopo } j)$$

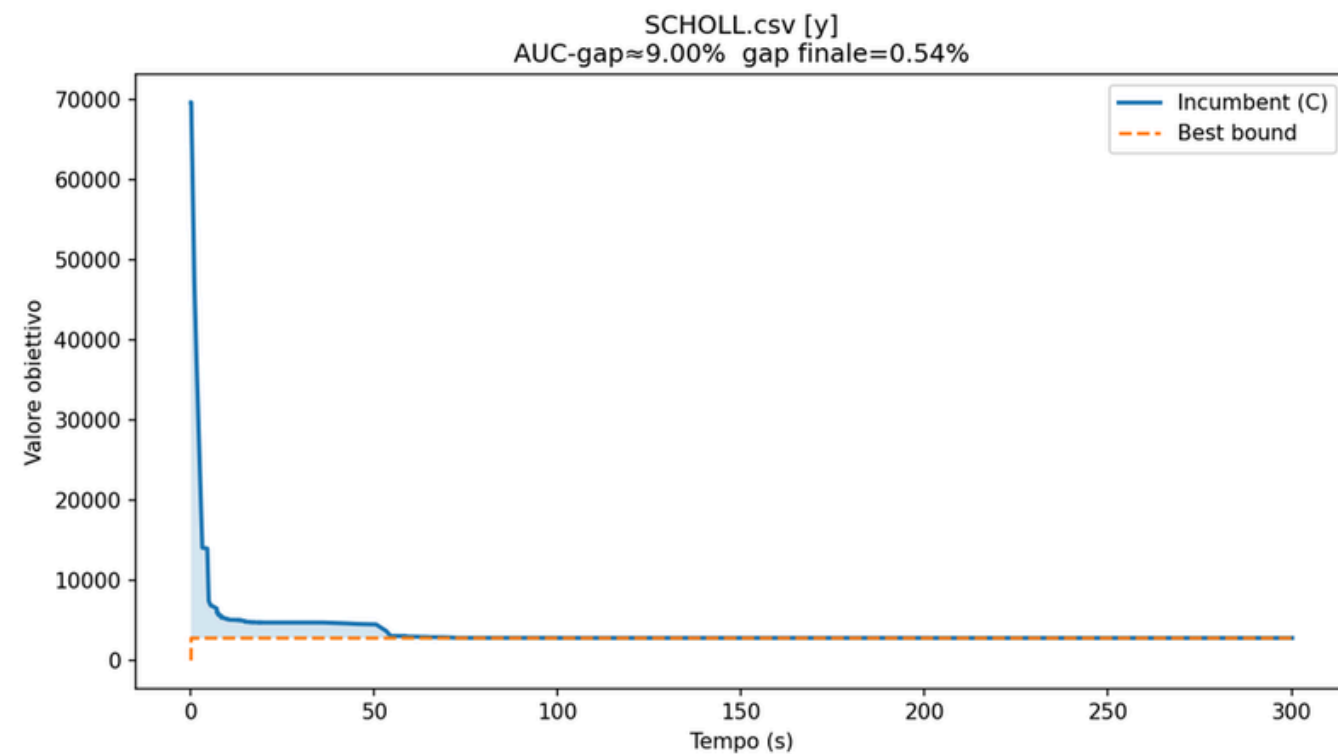
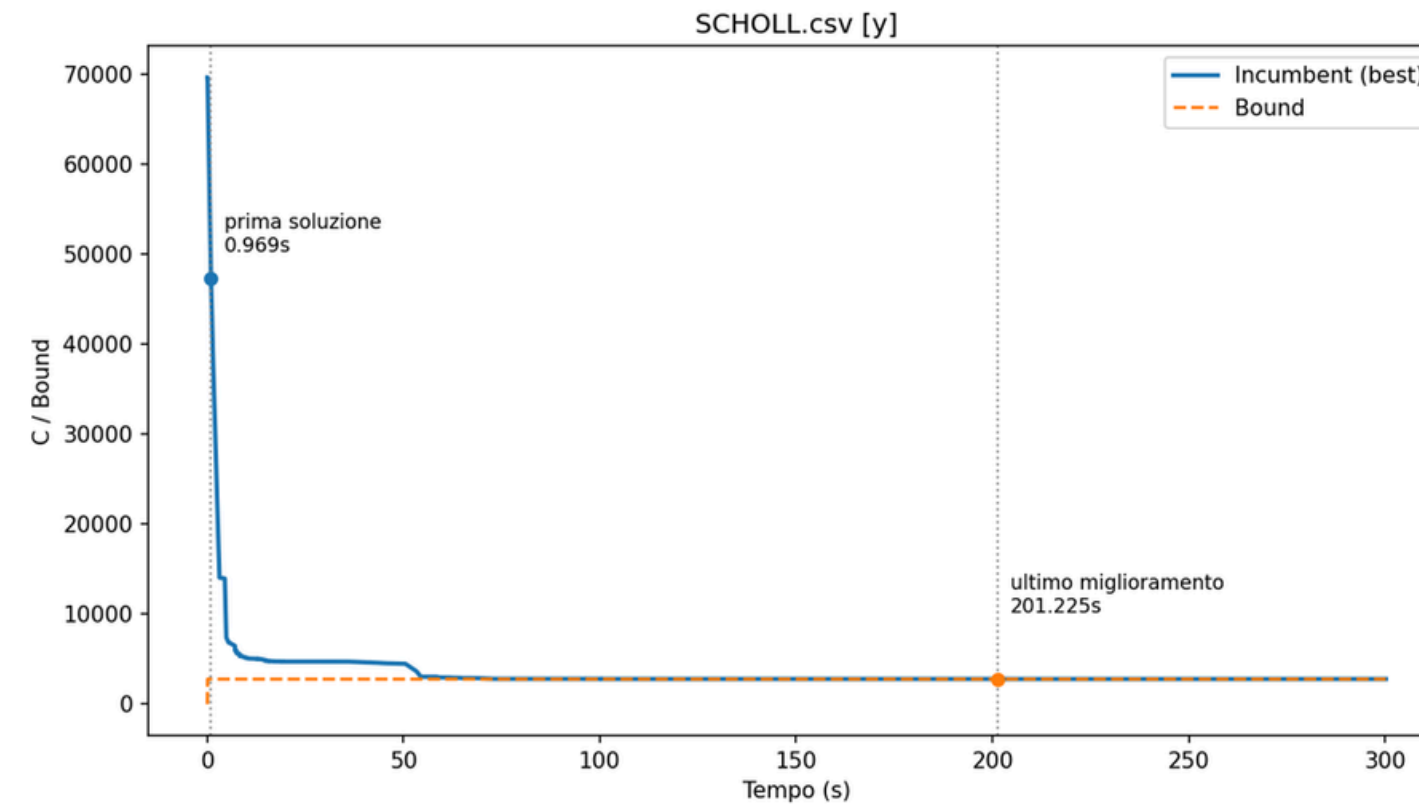
$$\sum_{i \in N} t_i x_{is} \leq C \quad \forall s \in S \quad (\text{limite di carico per stazione})$$

- N numero dei task
- S insieme di stazioni va da 1 a M
- t_i è un numero positivo che indica il tempo di completamento del task i
- l'insieme P contiene le precedenze ij

Differenze:

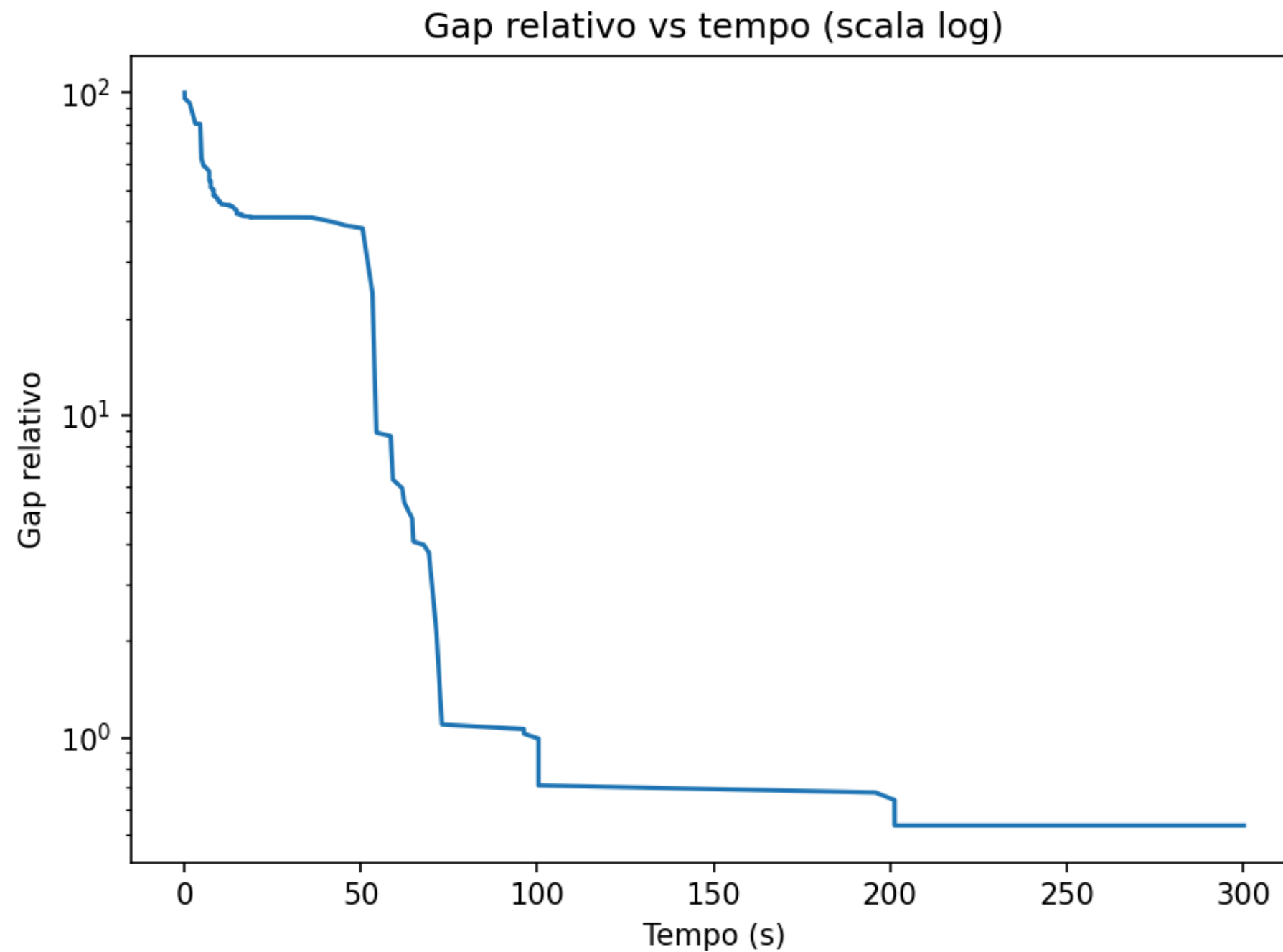
	Formulazione “prefix”	Formulazione “y”
Variabili	$x_{is} \in \{0, 1\}; u_{is} \in [0, 1]$ (prefisso: 1 se $\text{staz}(i) \leq s$); C	$x_{is} \in \{0, 1\}; y_i \in \{1, \dots, M\}$ (indice stazione); $L_s \geq 0$; C
Precedenze	$u_{is} \geq u_{js} \quad \forall (i \rightarrow j)$	$y_j \geq y_i$ (con link $y_i = \sum_s s x_{is}$)
Carico stazione	$\sum_i t_i x_{is} \leq C$ (senza L_s)	$L_s = \sum_i t_i x_{is}$, poi $L_s \leq C$
Forza del rilassamento LP	Più forte (le precedenze valgono a ogni stazione via (u))	Più debole (con (y) frazionabile in LP; impone solo media/indice)

Formulazione y



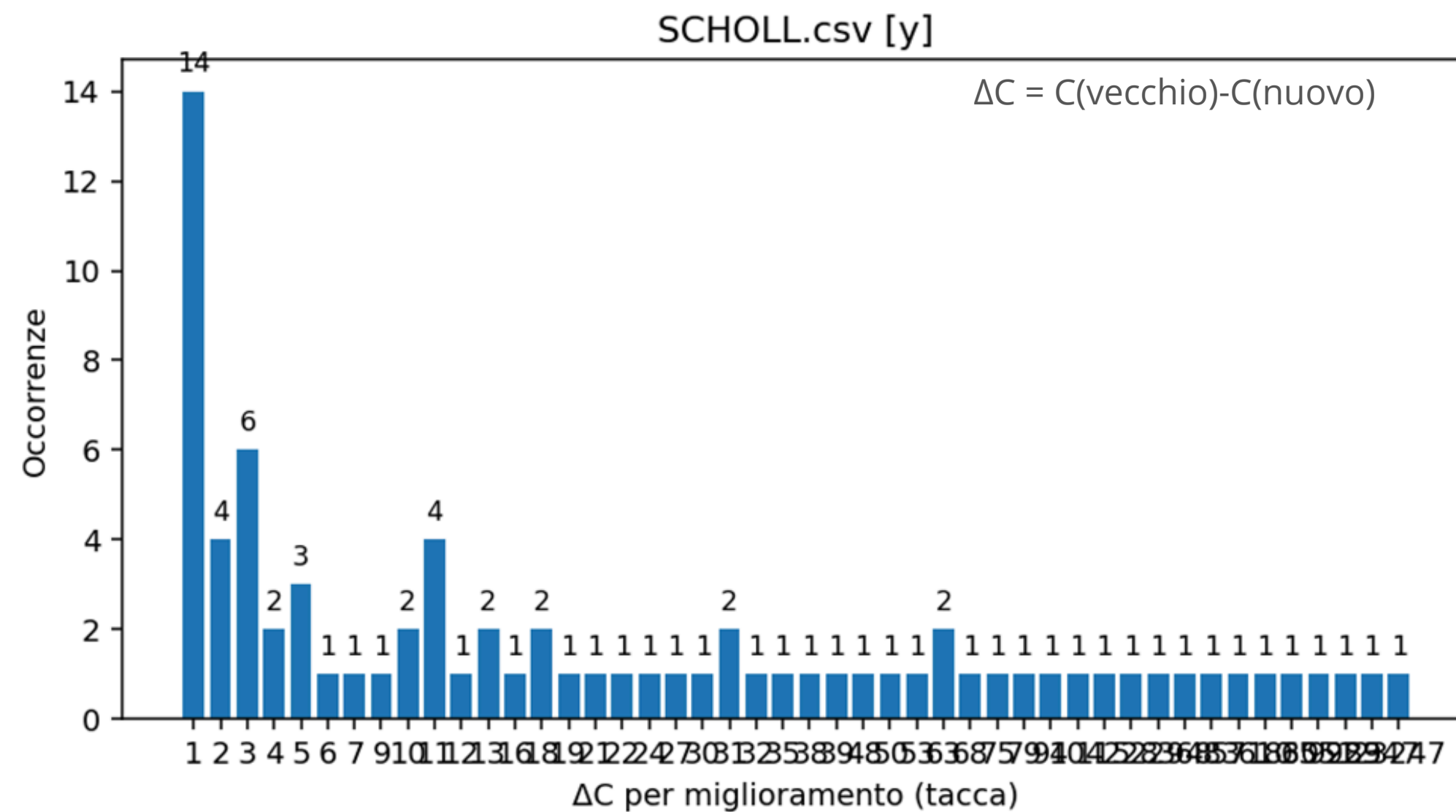
- Grafico Incumbent (C) vs Best bound: grande calo iniziale dell'incumbent (primi secondi) e bound che sale rapidamente → il gap crolla subito.
- Dopo ~50–60s le due curve quasi coincidono: la ricerca entra in fase di certificazione (tanto tempo speso a chiudere gli ultimi decimali).
- Gap finale 0.54%: non completamente chiuso, ma ottimi risultati.
- AUC-gap ≈ 9%: media temporale del gap lungo l'orizzonte → misura quanto a lungo la soluzione è stata lontana dall'ottimo; valore basso = convergenza rapida e stabile.

Formulazione y



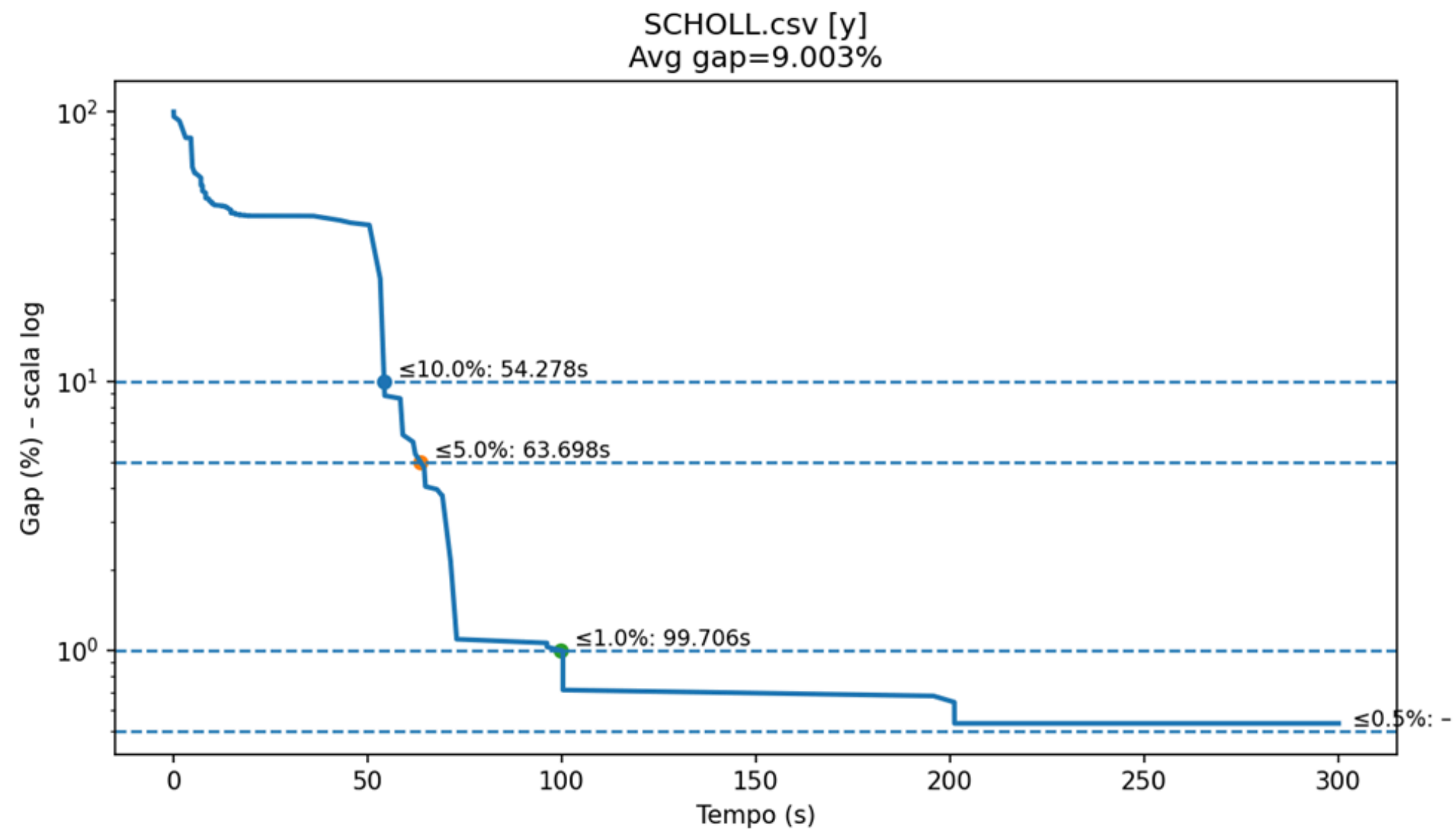
- Discesa rapida iniziale, poi rallenta: tipico dei MIP (facile migliorare all'inizio, difficile chiudere gli ultimi punti percentuali).
- Mostra il livello di gap raggiunto a fine orizzonte: evidenzia il punto in cui scendi $<1\%$ (se avviene) e se arrivi a $\text{gap} \approx 0$.

Formulazione y



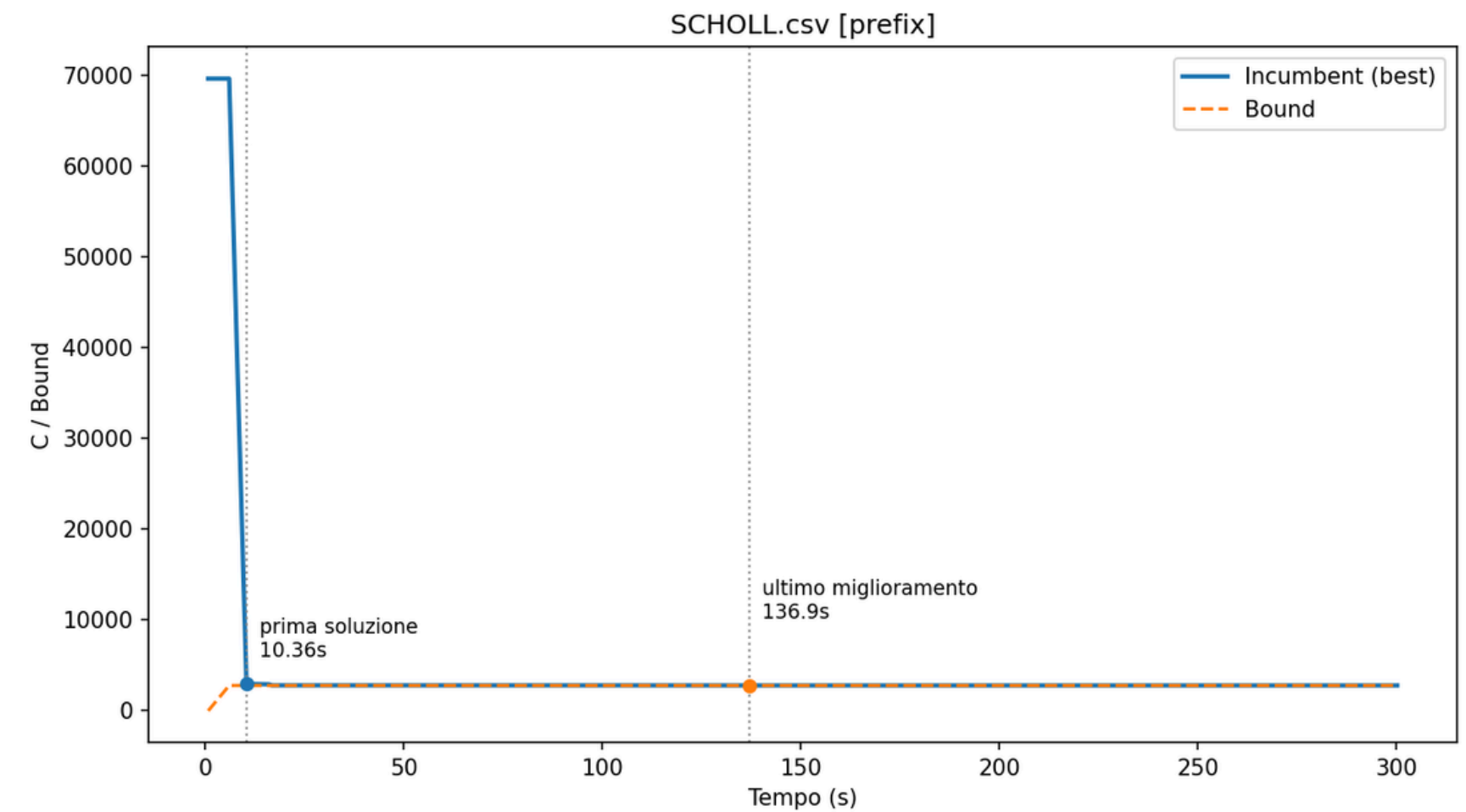
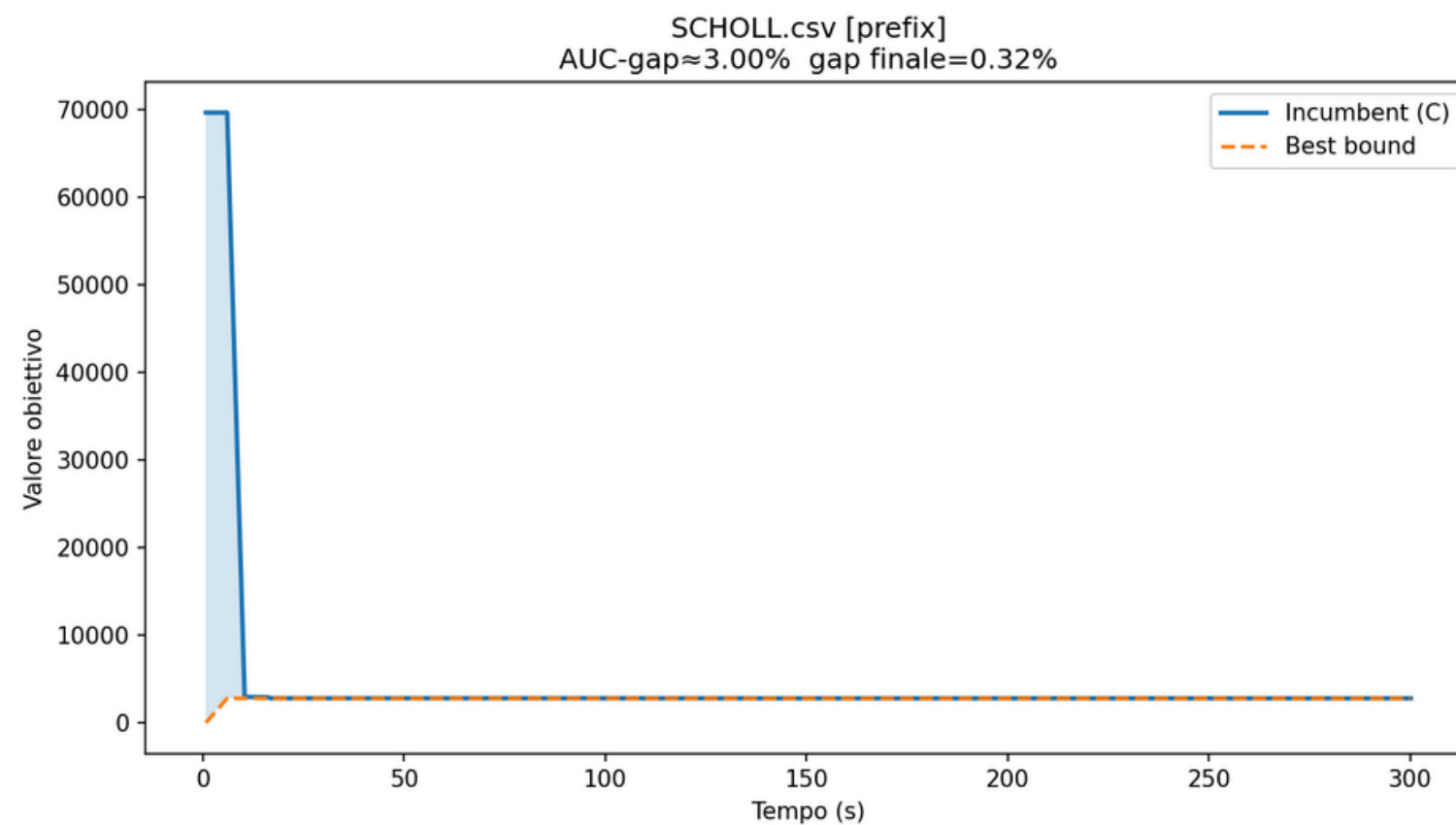
- Molti $\Delta C=1$: il solver rifinisce la soluzione vicino all'ottimo (micro-miglioramenti).
- Pochi ΔC grandi: i passi grossi avvengono all'inizio; poi prevale il perfezionamento.
- Messaggio: la formulazione porta a carichi molto bilanciati e miglioramenti finali "di fino".

Formulazione y



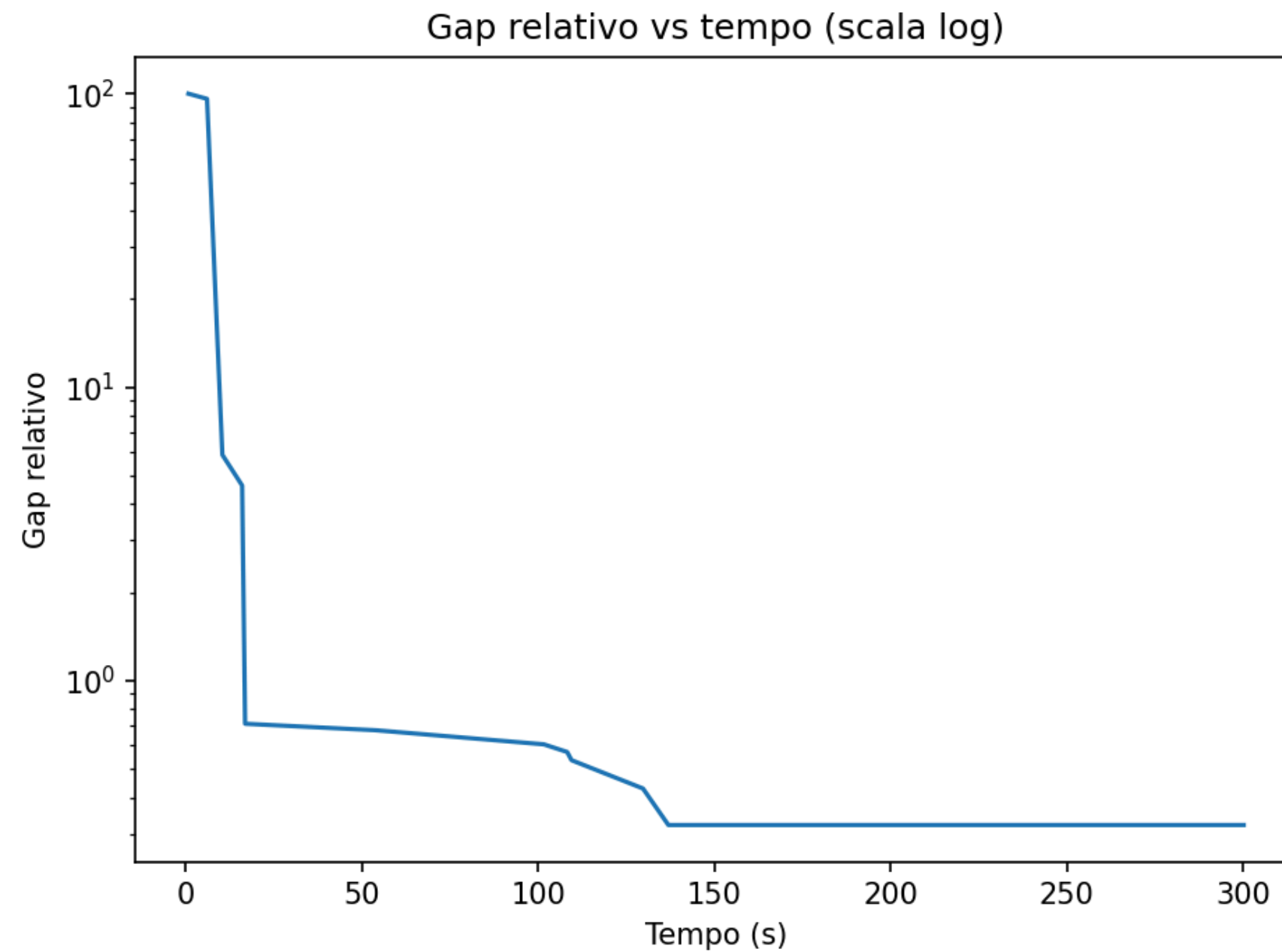
- Riassume quanto tempo serve per livelli di qualità standard: es. $\leq 10\%$ in $\sim 54s$, $\leq 5\%$ in $\sim 64s$, $\leq 1\%$ in $\sim 100s$ (numeri dai tuoi marker).
- Se $\leq 0.5\%$ non raggiunto, dillo: “entro 300s non scende sotto 0.5%” $\Rightarrow$ utile per decidere time-limit realistici.

Formulazione prefix



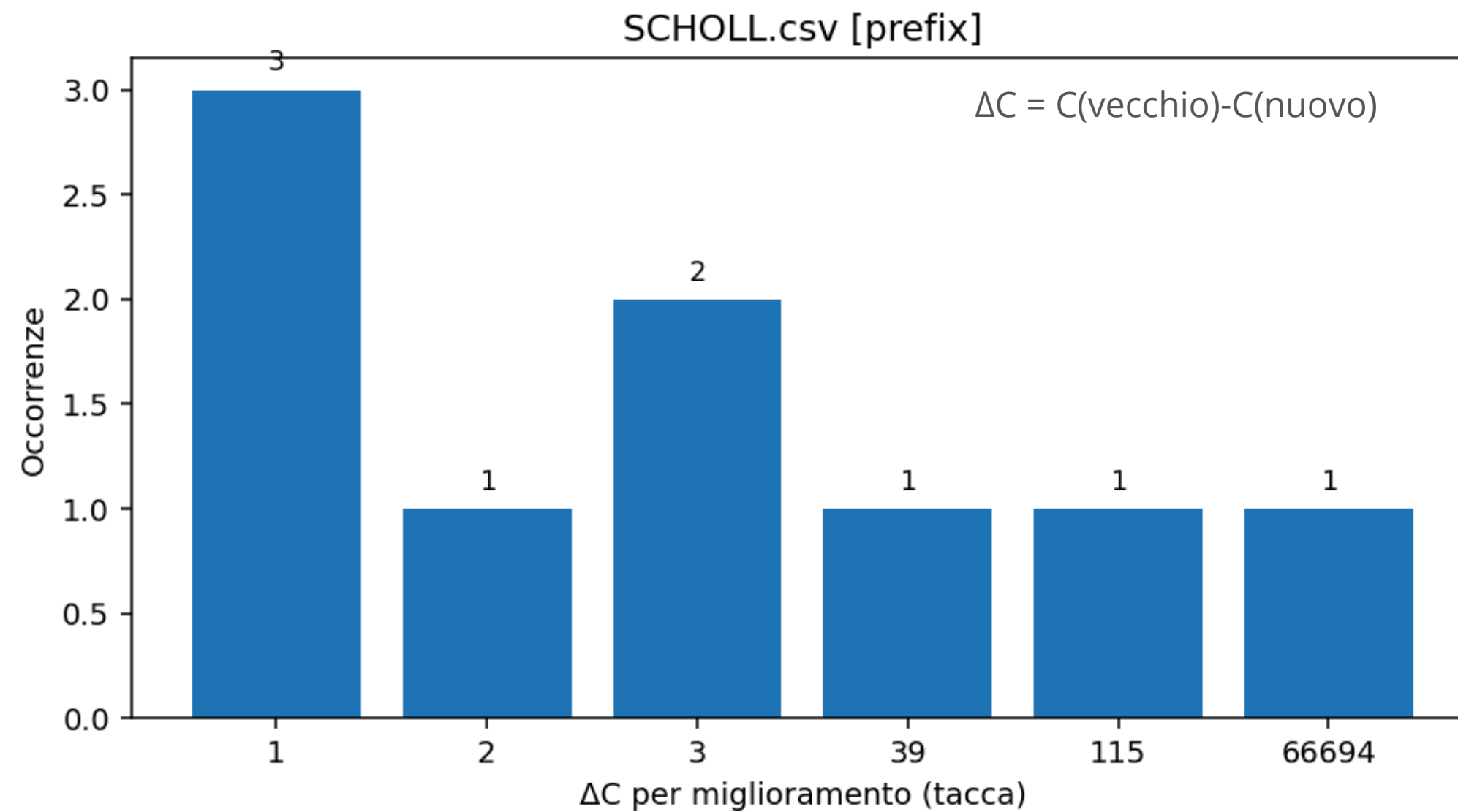
- Prima soluzione a ~ 10.36 ; subito **crollo** enorme dell'**incumbent**.
- Bound risale molto in fretta e resta sovrapposto alla curva blu per quasi tutto l'orizzonte $\rightarrow$ LP forte.
- AUC-gap $\approx 3\%$, gap finale $\approx 0.32\%$: media temporale del gap molto bassa $\rightarrow$ convergenza rapida e stabile.
- Il **solver** lavora quasi sempre **vicino all'ottimo**.

Formulazione prefix



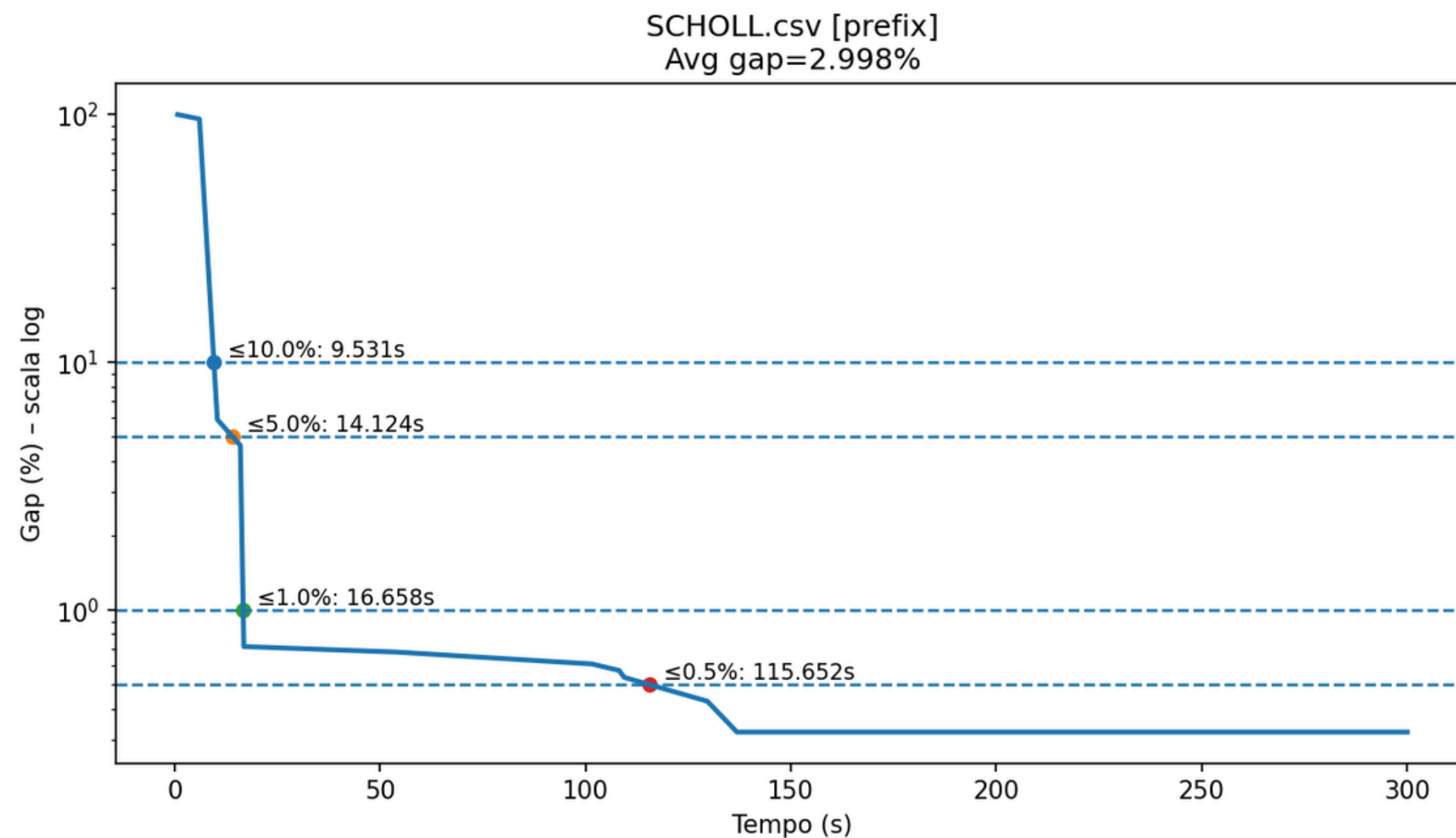
- Caduta rapidissima: da $>100\%$ a $<1\%$ in $\sim 17s$; poi discesa più lenta fino a $<0.5\%$.
- Profilo tipico di una formulazione più stretta: poco tempo per avvicinarsi all'ottimo, più tempo per certificarlo.

Formulazione prefix



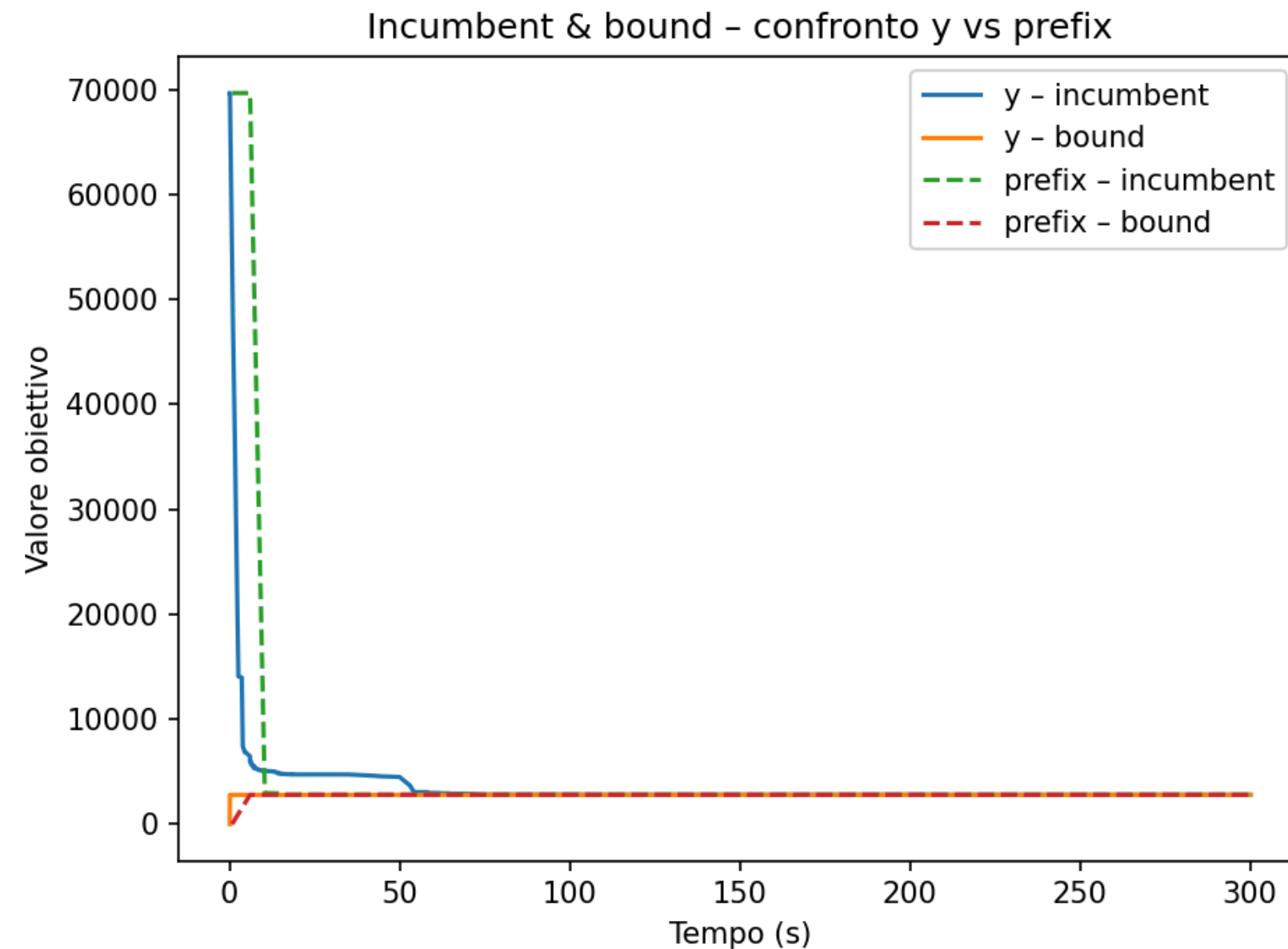
- Prevalgono i $\Delta C=1$ quindi il solver rifinisce in modo molto fine.
- All'inizio però ci sono pochi miglioramenti molto grandi (39, 115, 66694), che corrispondono ai grossi salti dal bound iniziale alla prima buona soluzione.

Formulazione prefix



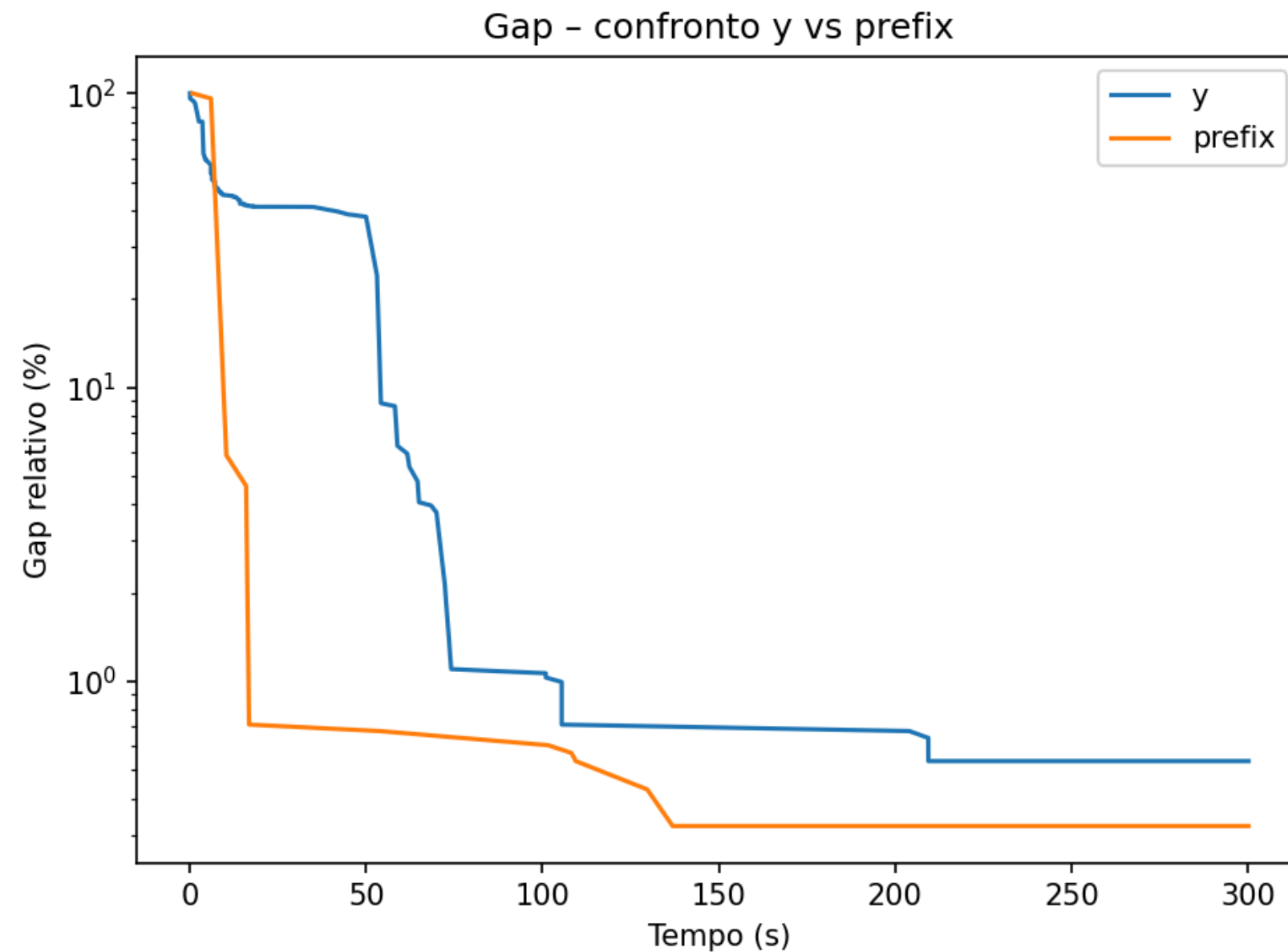
- $\leq 10\%$ in $\sim 9.5s$, $\leq 5\%$ in $\sim 14s$, $\leq 1\%$ in $\sim 16.65s$, $\leq 0.5\%$ in $\sim 115s$.
- Questi tempi sono nettamente migliori della formulazione y sullo stesso dataset (y scende $\leq 1\%$ $\sim 100s$ e non raggiunge 0.5% nell'orizzonte), evidenziando la forza del prefix.

Confronti



- In entrambi i casi c'è un forte miglioramento iniziale dell'incumbent: il valore di C crolla dai primi upper bound molto alti a valori vicini a quelli finali in pochi secondi.
- Dopo questa fase, le curve degli incumbent di y e prefix sono quasi sovrapposte: le due formulazioni trovano soluzioni di qualità molto simile.
- La differenza è nei bound: per prefix il bound raggiunge subito l'area dell'incumbent, mentre per y resta più distante a lungo.
- → prefix è quindi preferibile non tanto per la soluzione finale, quanto per la velocità con cui certifica la bontà della soluzione (gap più piccolo per tutto l'orizzonte di tempo).

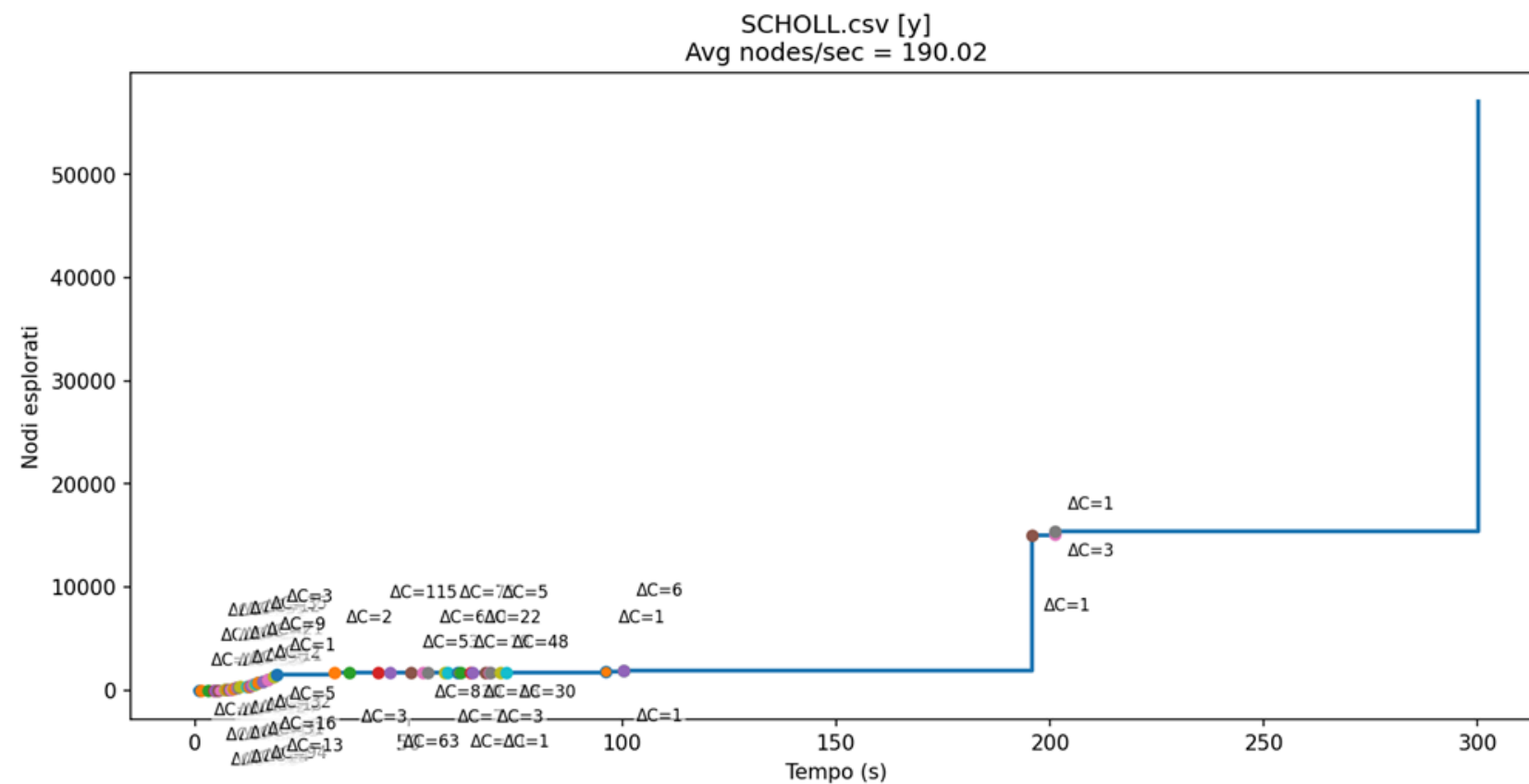
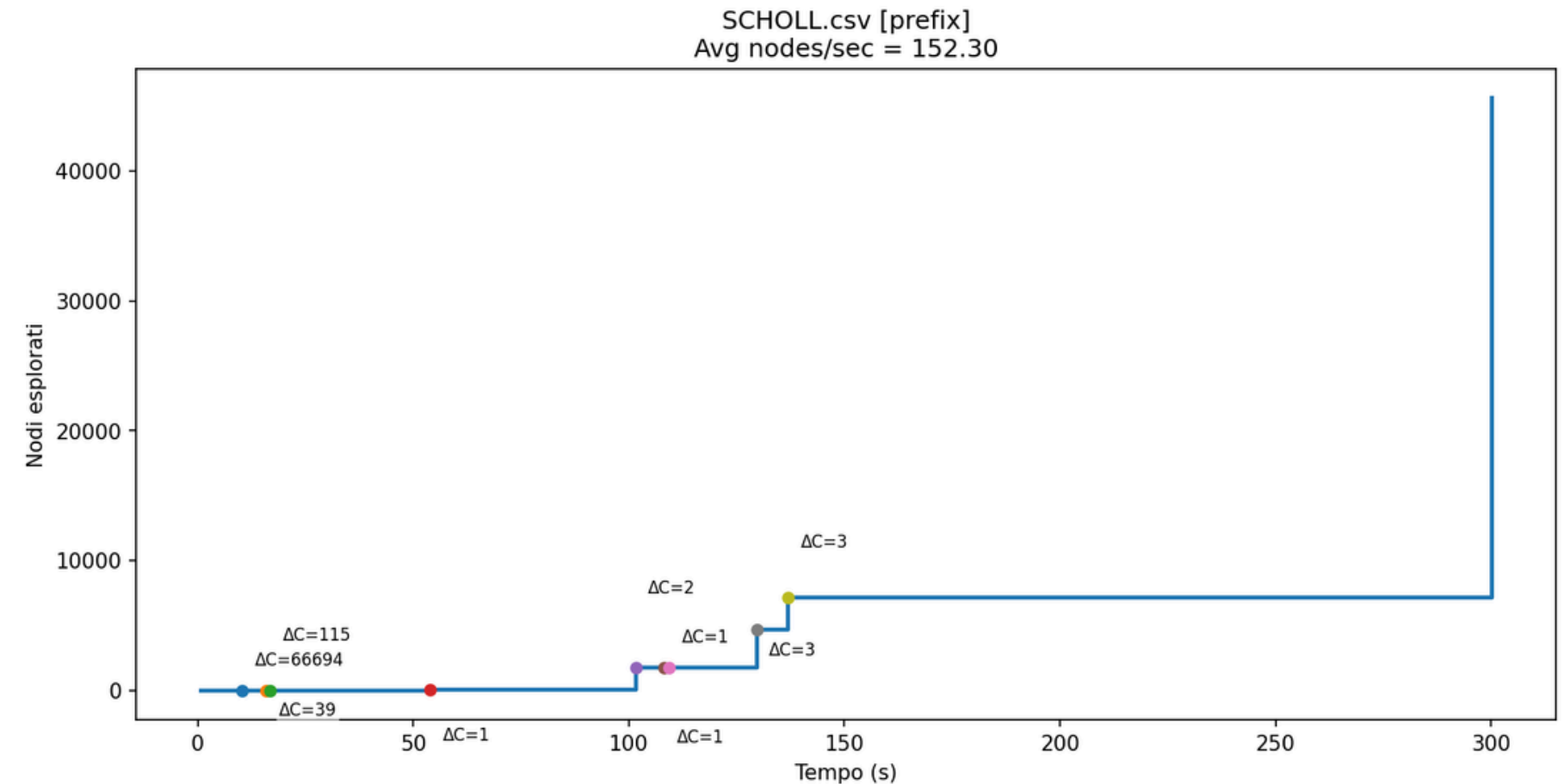
Confronti



- Entrambe le formulazioni partono con un gap molto alto ($>100\%$), ma prefix lo riduce drasticamente in pochissimi secondi: scende sotto l'1% già intorno ai 10–15s.
- La formulazione y chiude il gap molto più lentamente: impiega circa 90s per scendere sotto l'1% e poi rimane su valori leggermente peggiori.
- Conclusione: prefix fornisce un bound molto più forte e più rapido, arrivando sia prima sia più vicino all'ottimo (gap finale $\approx 0.3\%$ contro $\approx 0.5\%$ di y).

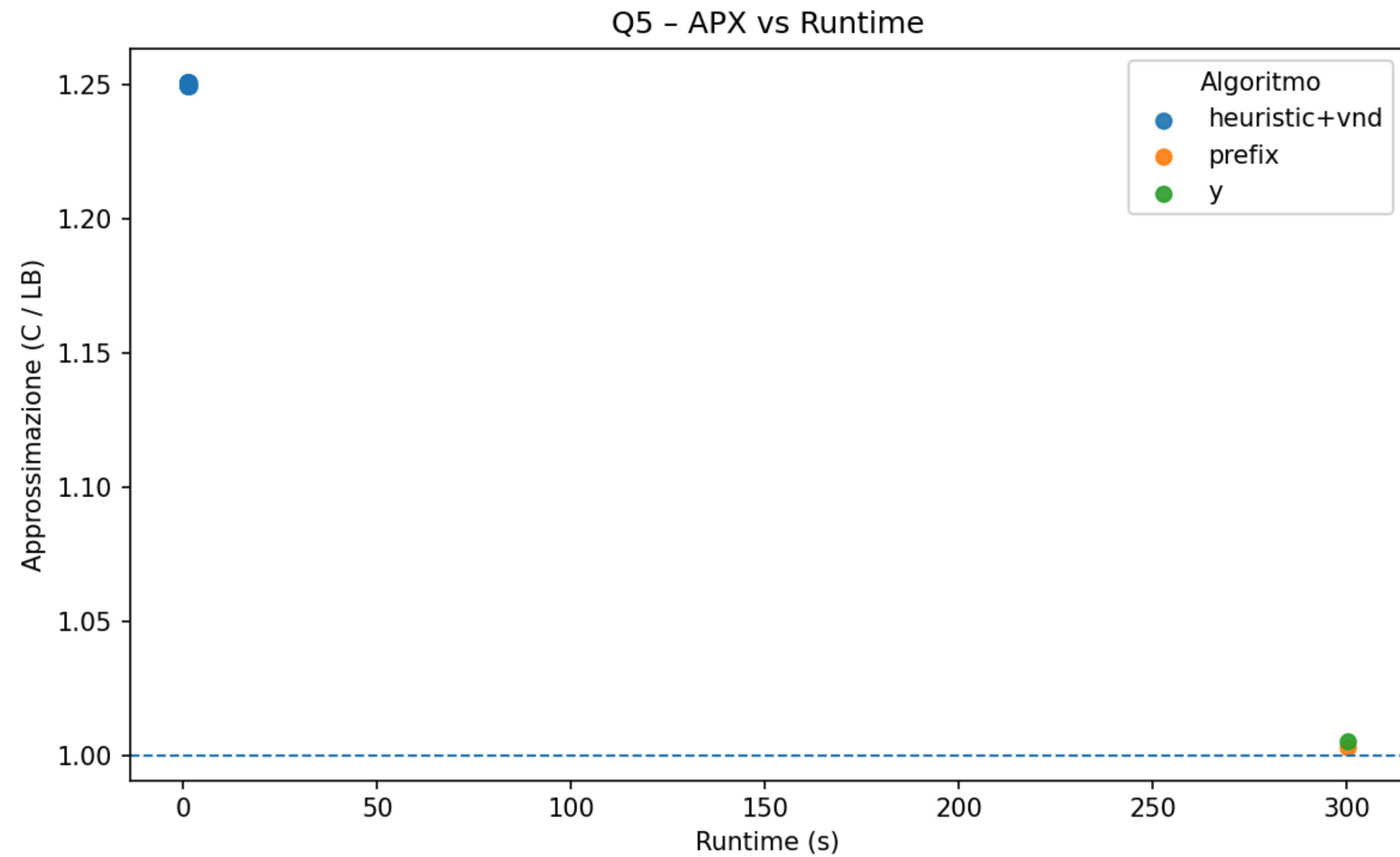
Confronti

- La curva a gradini mostra lunghi tratti quasi orizzontali (poche esplorazioni), intervallati da salti verticali.
- All'inizio bastano pochi nodi per ottenere miglioramenti molto grandi ($\Delta C = 66694, 115, 39$);
- man mano che ci si avvicina all'ottimo, servono molti più nodi per miglioramenti piccoli ($\Delta C = 1-3$).
- Dopo l'ultimo miglioramento di C , il solver continua a esplorare tantissimi nodi senza più migliorare.



- Andamento a scalini: lunghi tratti senza migliorare, poi miglioramenti puntuali del valore obiettivo.
- Le etichette ΔC mostrano quanto migliora ogni volta: utile per capire se i progressi sono fini ($\Delta C=1$) o grossi ($\Delta C \gg 1$).
- Avg nodes/sec quantifica l'intensità della ricerca: utile per confronti tra istanze o formulazioni.

Confronti



Architetture usate:

DISPOSITIVO 1:

CPU i9 11900H - 16GB DDR4 3200MT/s Single Channel

```
== [prefix] ==  
Model vars=14851 (bin=7425, int=7425) constrs=18322  
Status      : TIME_LIMIT  
Runtime (s): 300.098  
Nodi       : 89286  
Obj (C)    : 2796.0  
Best bound : 2787.0  
Gap finale : 0.3219%
```

```
== [y] ==  
Model vars=7748 (bin=7425, int=7722) constrs=1067  
Status      : TIME_LIMIT  
Runtime (s): 300.215  
Nodi       : 58678  
Obj (C)    : 2802.0  
Best bound : 2787.0  
Gap finale : 0.5353%
```

DISPOSITIVO 2:

CPU i7 14700HX - 16GB RAM DDR5 5600MT/s Dual Channel

```
== [prefix] ==  
Model vars=14851 (bin=7425, int=7425) constrs=18322  
Status      : TIME_LIMIT  
Runtime (s): 300.053  
Nodi       : 63009  
Obj (C)    : 2799.0  
Best bound : 2787.0  
Gap finale : 0.4287%
```

```
== [y] ==  
Model vars=7748 (bin=7425, int=7722) constrs=1067  
Status      : TIME_LIMIT  
Runtime (s): 300.038  
Nodi       : 201926  
Obj (C)    : 2799.0  
Best bound : 2787.0  
Gap finale : 0.4287%
```

SALBP-2

ALESSANDRO LORI - GIADA PICA

Grazie dell'attenzione

